Software Reengineering

National University of Computer and Emerging Sciences, Islamabad

by

Kashaf Fatima 22i-2415

Safi ur Rehman 22i-2534

Amna Noor 22i-1529

Assignment # 3

Submitted to [Ma'am Nigar Azhar]

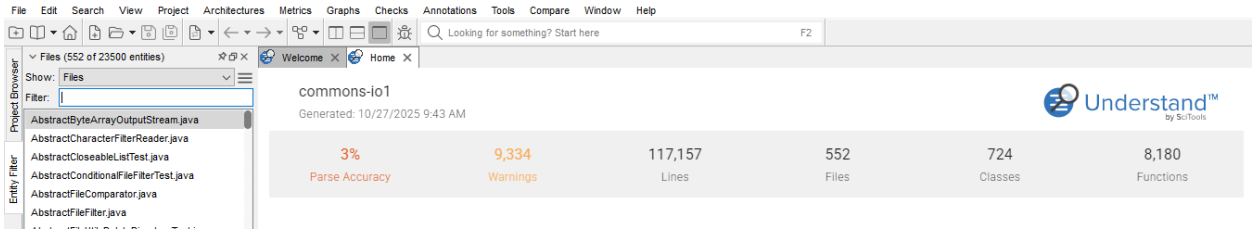
Date: 10/28/2025

Table of Contents

1. Project Overview

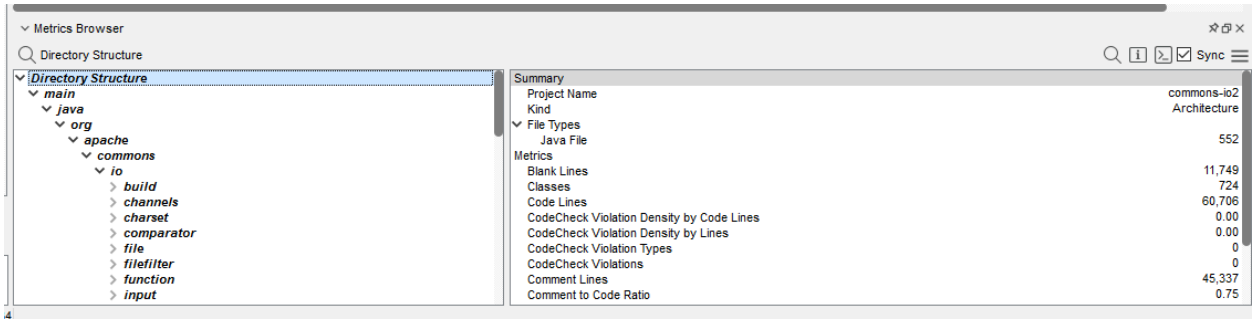
Code Files

Total number of code files = 552



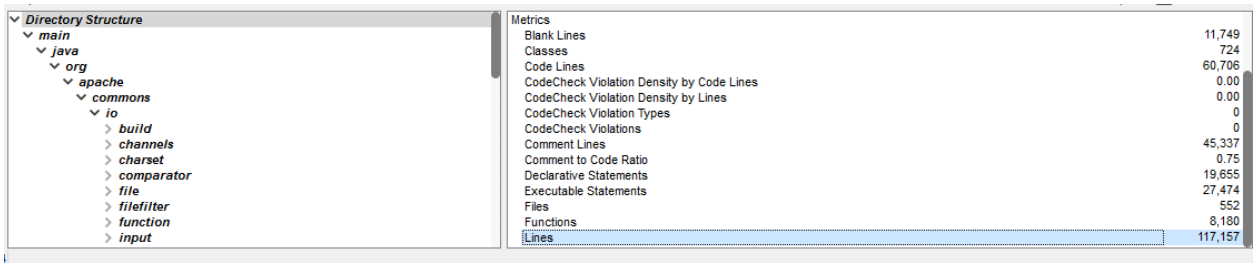
Java Files

This repository contains java files. Number of java files = 552



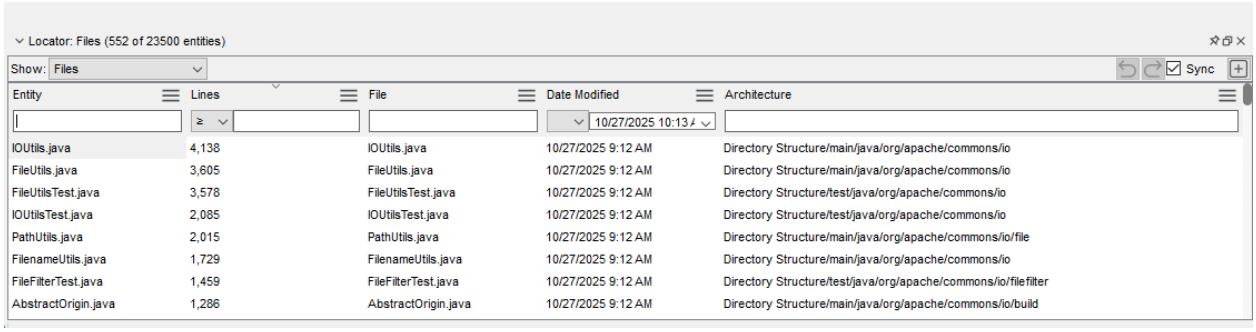
Lines of code

Total number code lines = 117,129



Highest LOC

IOUtiles.java has the highest number of lines i.e., 4125 LOC



Entity	Lines	File	Date Modified	Architecture
	≥		10/27/2025 10:13 A	
IOUtils.java	4,138	IOUtils.java	10/27/2025 9:12 AM	Directory Structure/main/java/org/apache/commons/io
FileUtils.java	3,605	FileUtils.java	10/27/2025 9:12 AM	Directory Structure/main/java/org/apache/commons/io
FileUtilsTest.java	3,578	FileUtilsTest.java	10/27/2025 9:12 AM	Directory Structure/test/java/org/apache/commons/io
IOUtilsTest.java	2,085	IOUtilsTest.java	10/27/2025 9:12 AM	Directory Structure/test/java/org/apache/commons/io
PathUtils.java	2,015	PathUtils.java	10/27/2025 9:12 AM	Directory Structure/main/java/org/apache/commons/io/file
FilenameUtils.java	1,729	FilenameUtils.java	10/27/2025 9:12 AM	Directory Structure/main/java/org/apache/commons/io
FileFilterTest.java	1,459	FileFilterTest.java	10/27/2025 9:12 AM	Directory Structure/test/java/org/apache/commons/io/filefilter
AbstractOrigin.java	1,286	AbstractOrigin.java	10/27/2025 9:12 AM	Directory Structure/main/java/org/apache/commons/io/build

funcUon

● IOUtiles.java's role in the project:

A utility class for managing different input and output stream operations is provided by this file. It comes with a number of static methods that make common I/O chores easier.

The following features are offered by the class:

- close Silently: Closes streams safely, even if they are null, without raising exceptions.
- toread: Effectively retrieves information from input streams.
- The task of writing data to output streams is handled by write.
- Copy: Optimizes data transfers between streams.
- contentEquals: Verifies if the content in two streams is the same.

It also enables conversions between characters and bytes, enabling users to specify particular encodings as required.

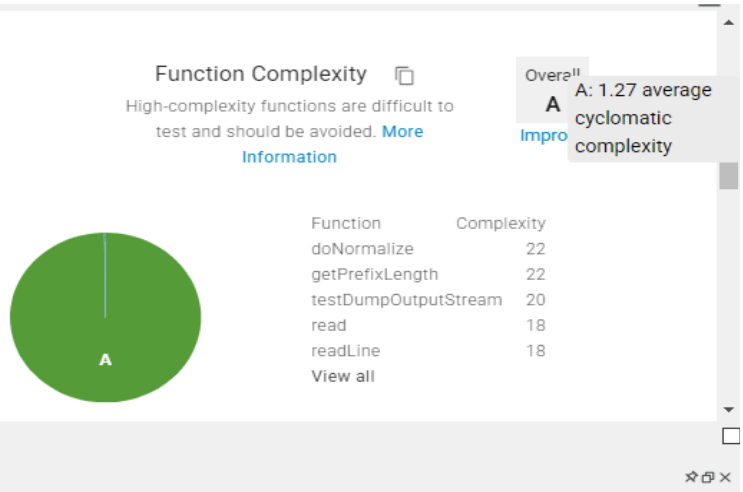
Average cyclomaUC complexity

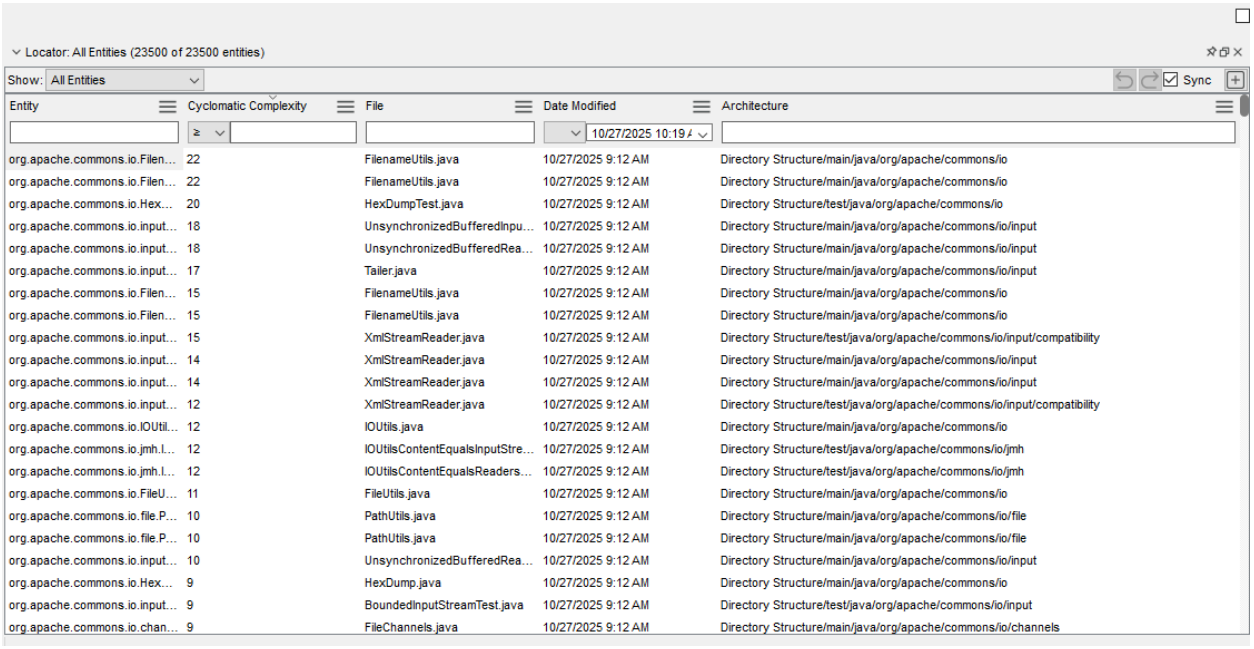
Total Complexity =10399 (sum of all files)

Total Functions = 8180

Average Complexity = Total Complexity / Total Functions

= 10399 / 8180 = **1.27**





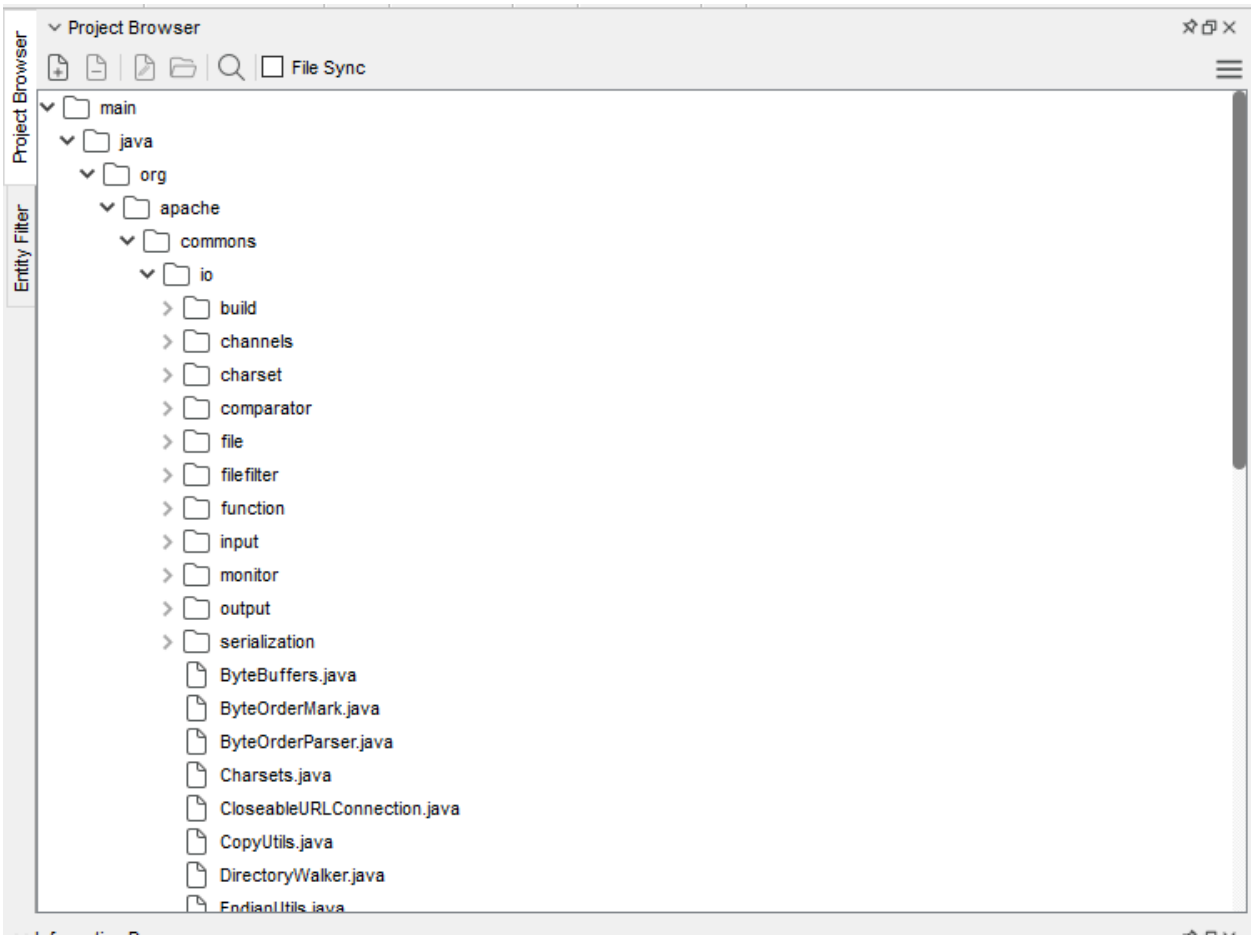
Comparison with recommended threshold

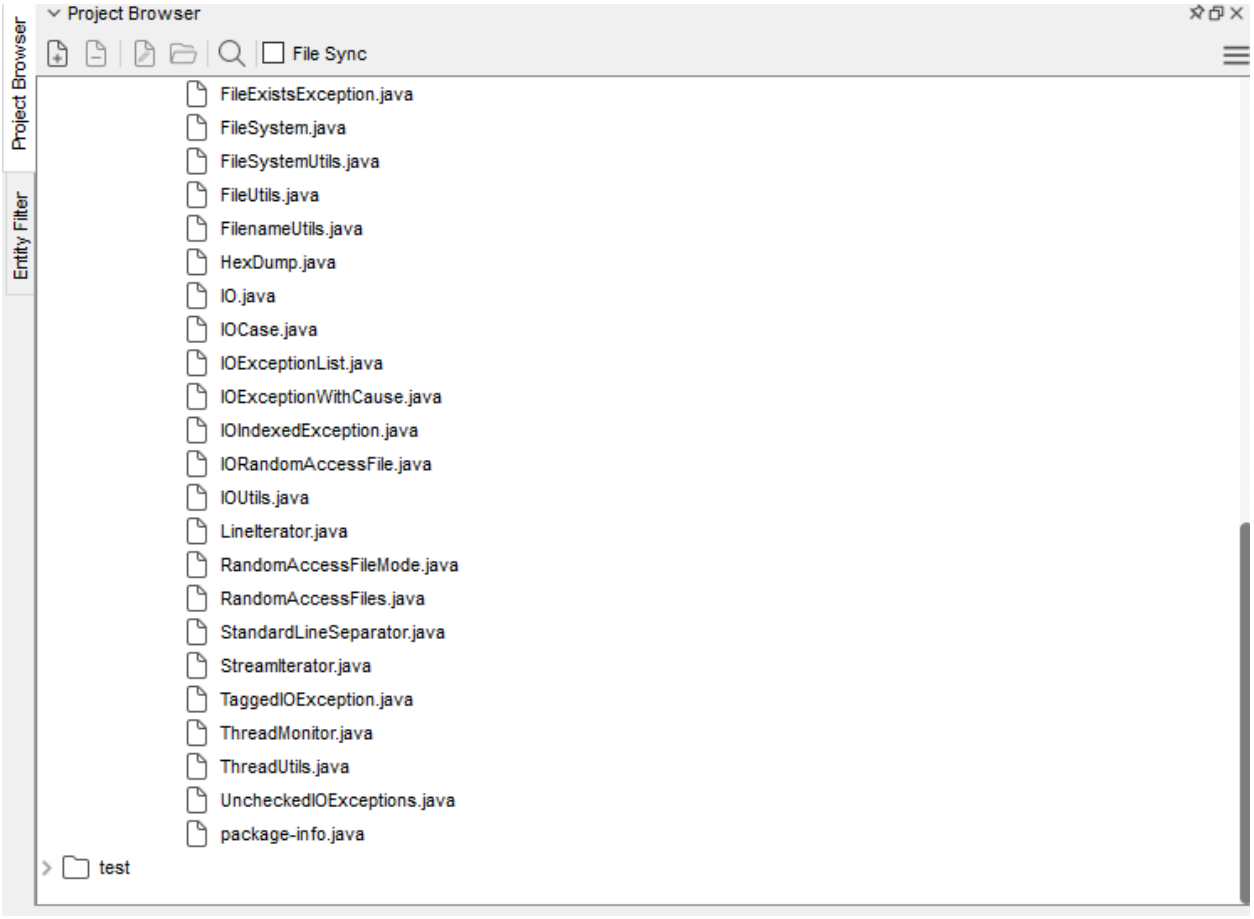
The average cyclomatic complexity of the project is 1.27, which is significantly lower than the suggested criterion of 10. This suggests that the source is highly readable and maintainable due to its excellent organization, ease of comprehension, and low level of logical complexity.

Structure of source code

1. The project has a root folder with `pom.xml` and well-organized subdirectories like `src/main/java` and `src/test/java`, which are part of the normal Maven directory structure.
2. Streams, files, filters, charsets, comparators, and other functions are used to organize the source files into packages.

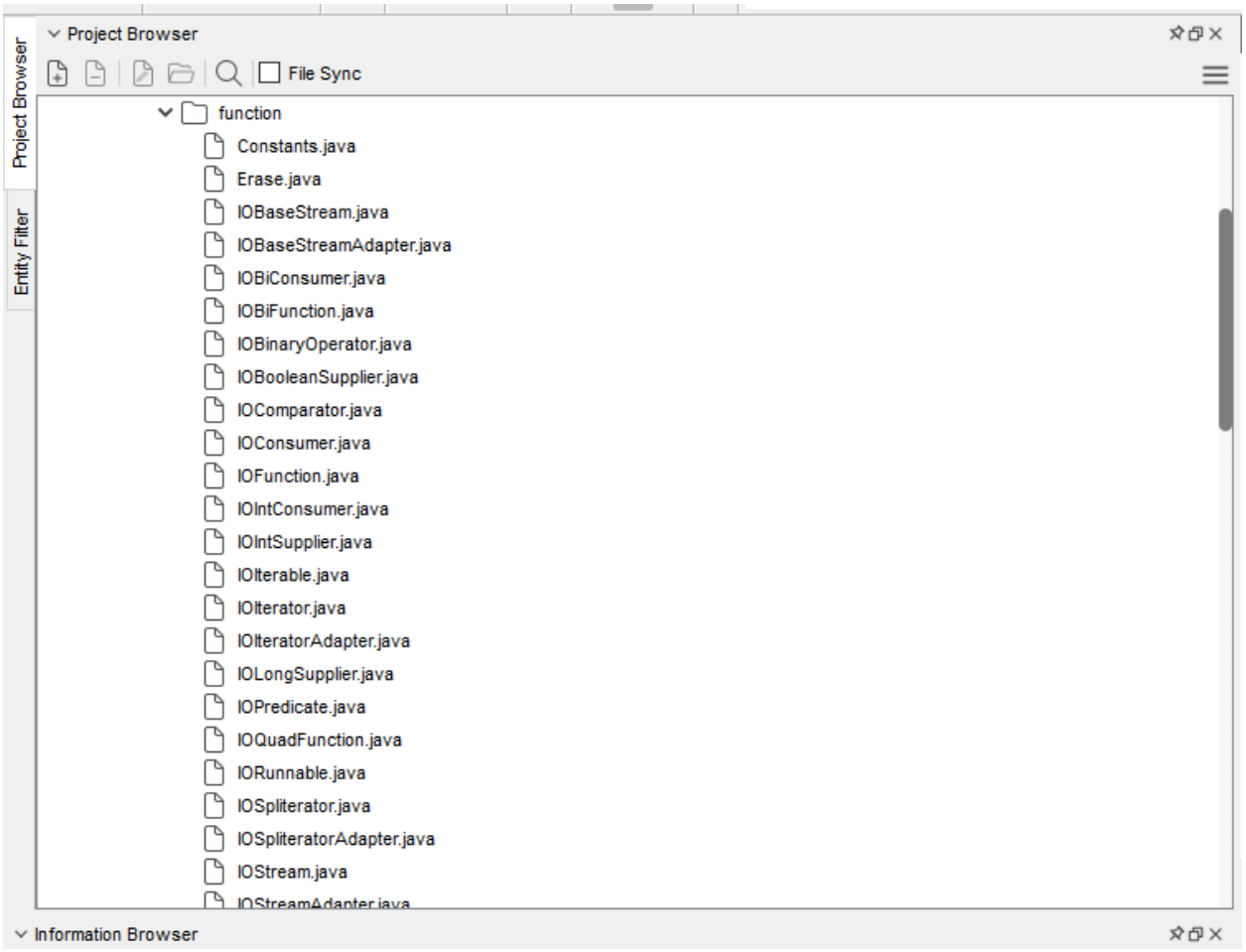
3. Documentation, licensing information, and contribution guidelines are among the important items that are properly situated at the project's root level.

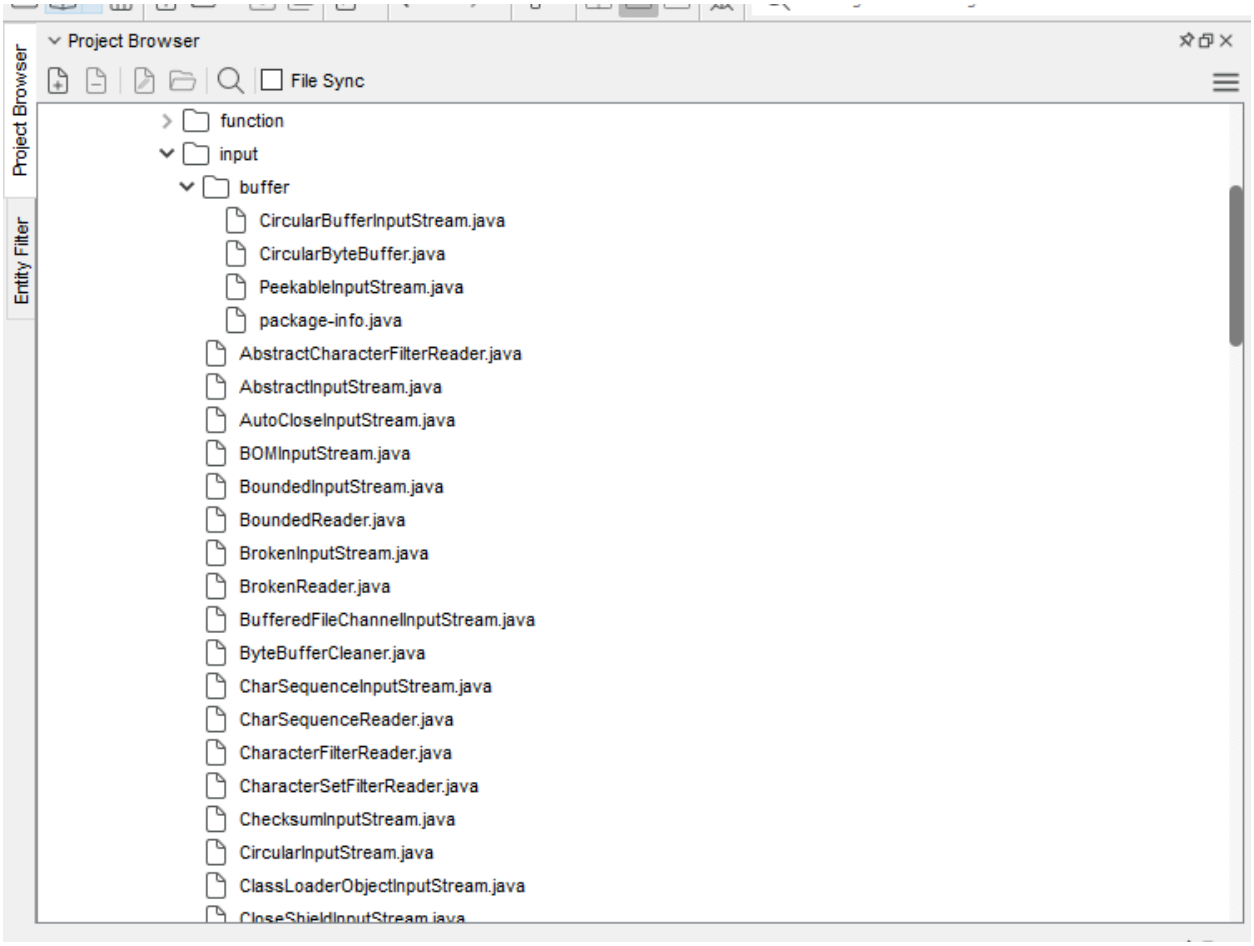




Organization of Files

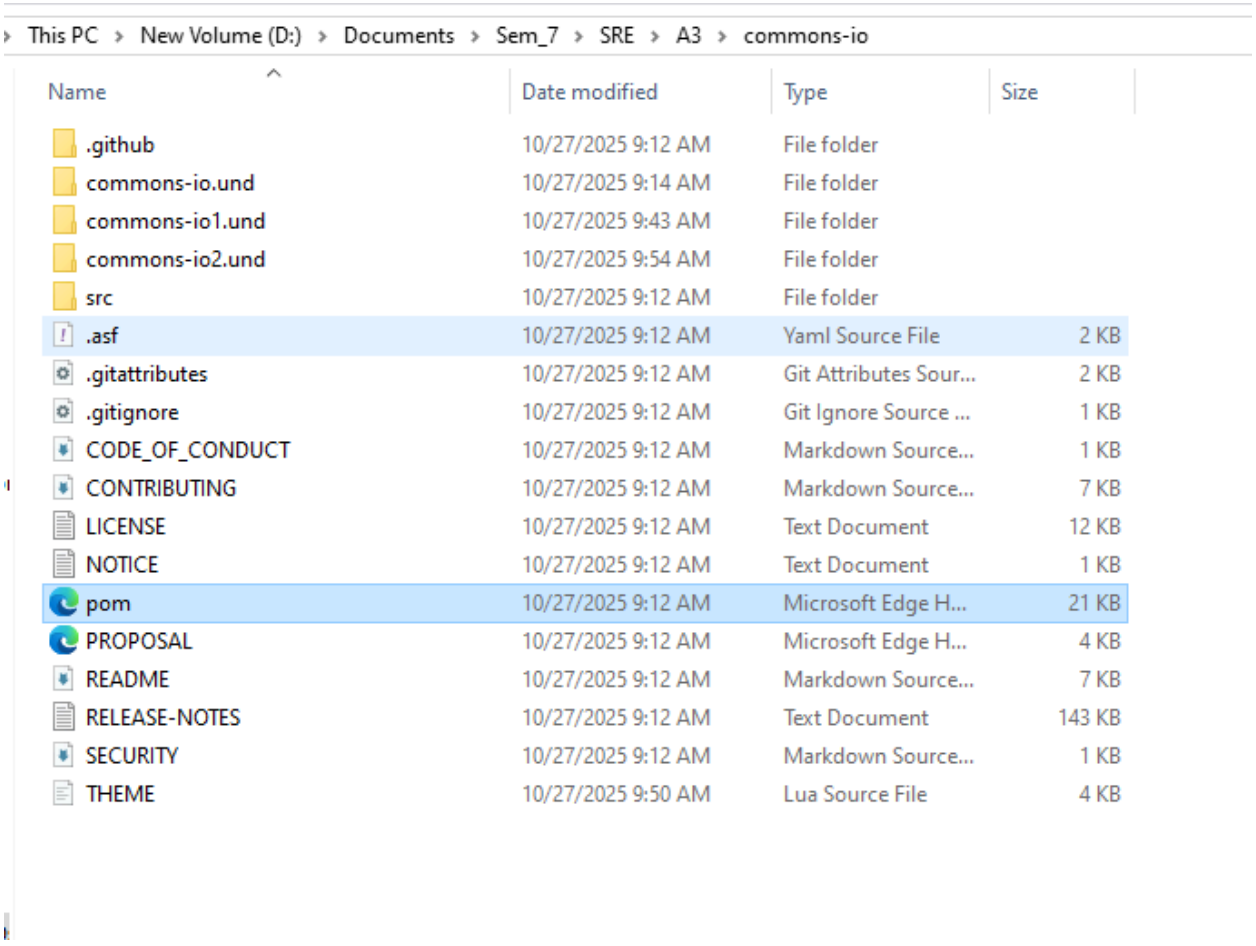
1. In accordance with accepted practices, test files are appropriately separated from the main source code and stored in a specific test directory.
2. The project structure is organized into several utility packages, each of which is in charge of a particular I/O function. These packages include input, output, filters, and file monitoring.
3. To improve organization, modularity, and ease of maintenance, functional files are kept in distinct directories.


Project Browser
Entity Filter
Project Browser
File Sync
function
Constants.java
Erase.java
IOBaseStream.java
IOBaseStreamAdapter.java
IOBiConsumer.java
IOBiFunction.java
IOBinaryOperator.java
IOBooleanSupplier.java
IOComparator.java
IOConsumer.java
IOFunction.java
IOIntConsumer.java
IOIntSupplier.java
IOIterable.java
IOIterator.java
IOIteratorAdapter.java
IOLongSupplier.java
IOPredicate.java
IOQuadFunction.java
IORunnable.java
IOSpliterator.java
IOSpliteratorAdapter.java
IOStream.java
Information Browser



Build Script

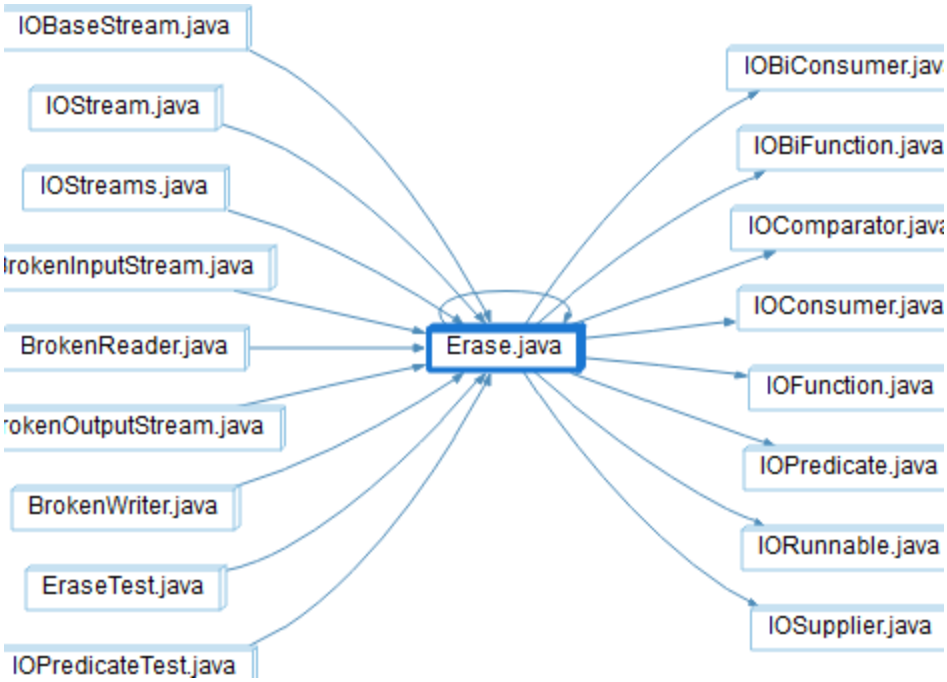
Yes, a build script called pom.xml is included in the project.
This file, which specifies the project's dependencies, plugins, and build lifecycle, acts as the Maven build configuration. It makes it possible for Apache Maven to effectively automate and manage processes like project compilation, testing, packaging, and deployment.



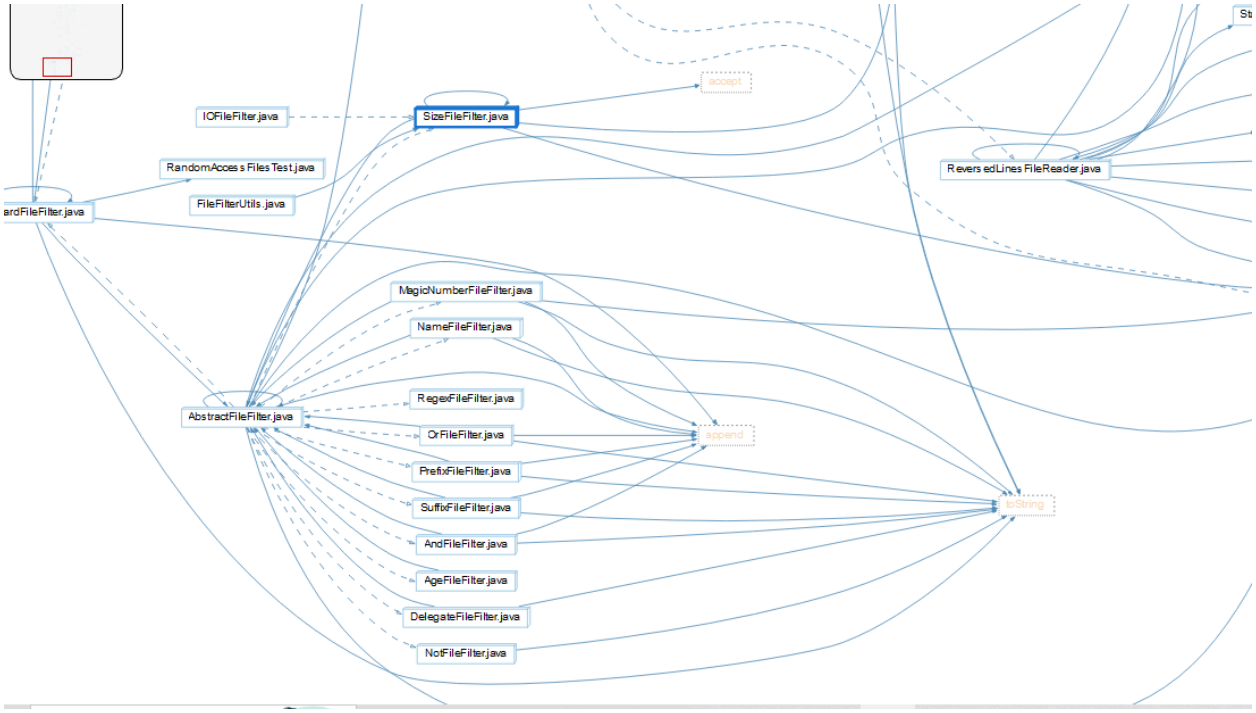
2. Architectural Analysis

Top 3 most coupled packages

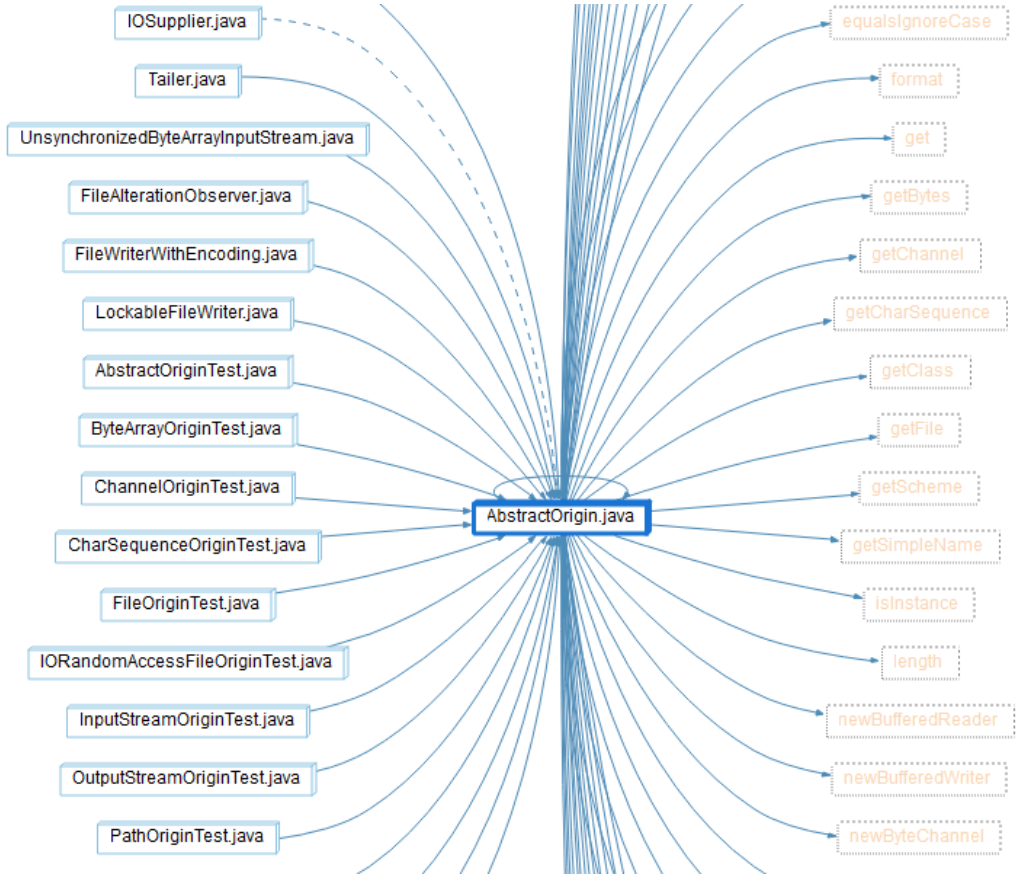
1. org.apache.commons.io.function



2. org.apache.commons.io.filefilter



3. org.apache.commons.io.build

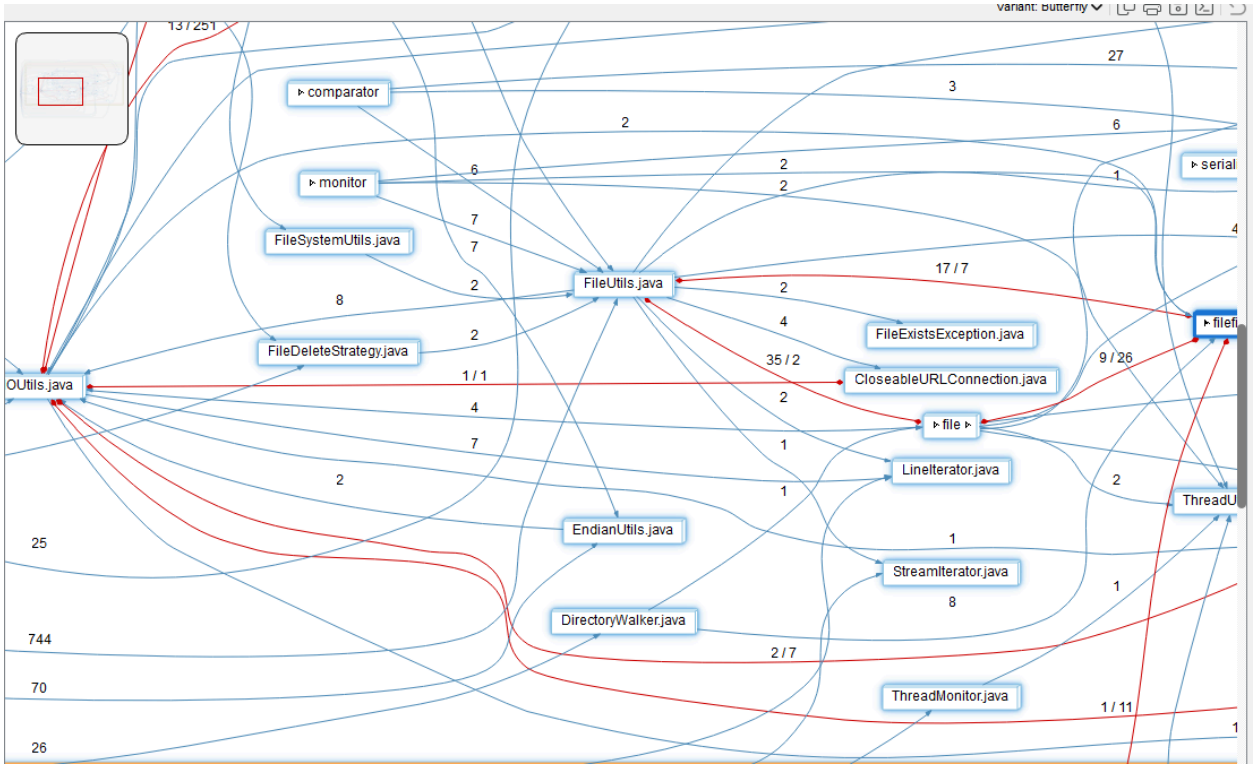


Circular Dependencies between packages

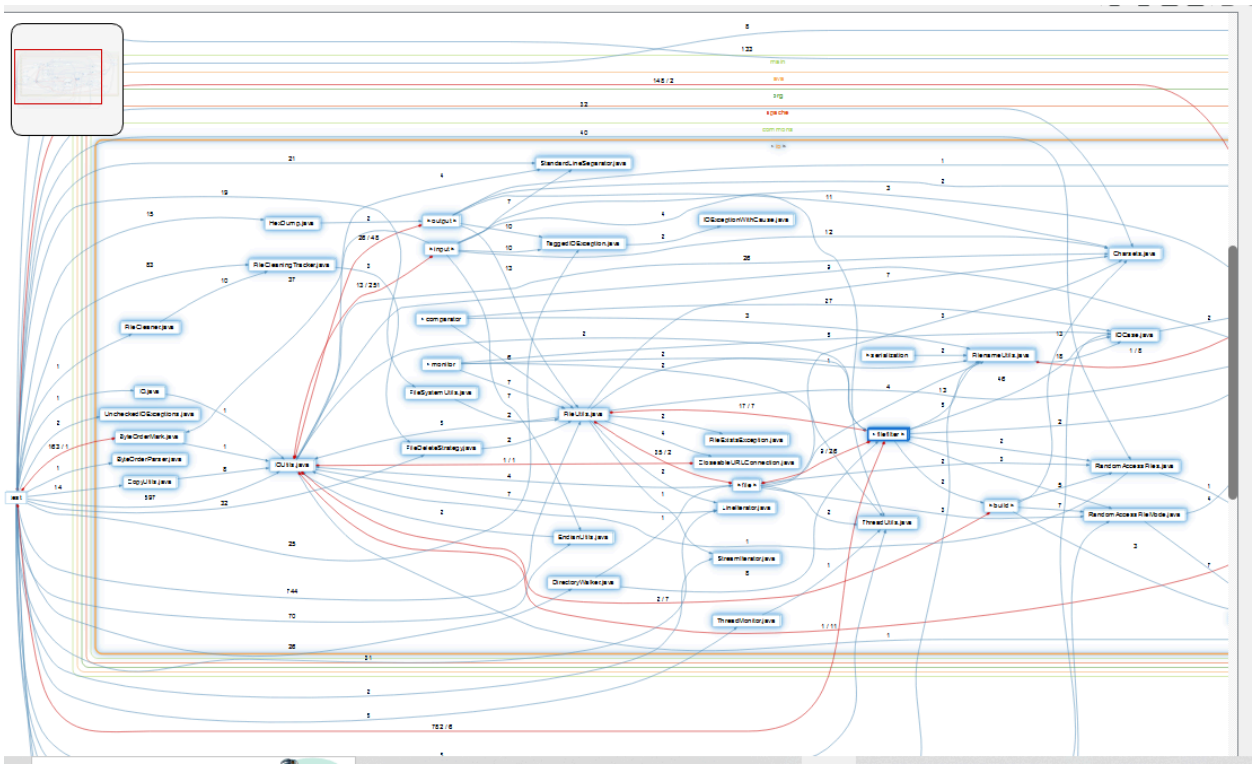
It was discovered that the input package and fielfilter had a cyclic dependency.

Reason: Data.repository uses input classes for validation, and fileutils imports models from data.repository.

Solution: To isolate validation logic from both packages, add a new layer (such as data.validator).



Visualization of Dependency Graph



Architectural Patterns

With IOUtils.java serving as the core hub linking to several specialized utility classes in charge of file operations, monitoring, and exception handling, the Apache Commons IO library has a hub-and-spoke class structure. This library is a low-level, reusable utility framework that prioritizes modularity and reusability over user interface separation, in contrast to MVC-based solutions.

Key Patterns:

- **Helper/Utility Pattern:**
  Static helper classes that provide common I/O functionalities, including IOUtils.java, FileUtils.java, and FilenameUtils.java, make up the library.
  Quality: Easy to use and reuse, although a little too centered on IOUtils.java (Score: 8/10).
- **Strategy Pattern:**
  This paradigm, which is illustrated in FileDeleteStrategy.java, promotes adaptable and extensible file-handling logic by enabling the use of interchangeable deletion algorithms.
  Quality: The product is loosely connected and highly expandable. (9 out of 10).
- **Observer Pattern:**
  To facilitate event-driven file change detection, the Observer Pattern is implemented in file monitoring classes such as FileCleaningTracker.java and FileAlterationObserver.java.
  Quality: Dependent on polling, which limits its effectiveness for real-time file monitoring. (Score: 8/10).

Design Principles Evaluation

OCP: Strongly adhered to, the Open/Closed Principle (OCP) allows for the extension of functionality without changing current code through the use of strategies and filters.
LSP & ISP: Consistent subclass implementations and clearly specified, targeted interfaces enable the effective use of the Liskov Substitution Principle (LSP) and Interface Segregation Principle (ISP).
SRP: Generally well-maintained, each class concentrates on a particular task (for example, FileUtils for file operations, FilenameUtils for path handling) in accordance with the Single

Responsibility Principle (SRP). However, because it takes on many tasks, IOUtils.java exhibits a minor infringement.
Some utilities continue to rely on concrete classes rather than abstractions, indicating that the **DIP:** Dependency Inversion Principle (DIP) is only partially implemented.
Overall Assessment: The architecture exhibits good extensibility, reusability, and modularity. To improve long-term maintainability, minor coupling surrounding IOUtils.java could be further reduced.

3. Code Quality Analysis

Functions with highest complexity

Top five most complex methods(using the Metrics Browser) are as follow

1. **FilenameUtils.java : 22 (getPrefixLength):**

   Reason:

   - manages many file path formats that are platform-specific (Windows, Unix, etc.).
   - includes numerous conditional tests (if, otherwise if, switch) to verify drive letters, UNC paths, and prefixes.
   - The total complexity rises with each condition, which adds a new decision branch.

2. **HexDumpTest.java : 20 (testDumpOutputStream):**

   Reason:

   - Multiple data dump verifications are carried out via a huge unit test approach.
   - contains several assertions, loops, and test cases all inside the same function.
   - Repetitive setup and validation logic causes test methods to frequently become complex.

3. **UnsynchronizedBufferedInputStream.java : 18 (readLine):**

   Reason:

   - reads lines from a stream without synchronization by implementing custom logic.
   - includes nested conditions and several loops to handle end-of-line characters (\n, \r\n, etc.).
   - The decision paths are further expanded by buffer management and error handling.

4. **UnsynchronizedBufferedReader.java : 18 (read):**

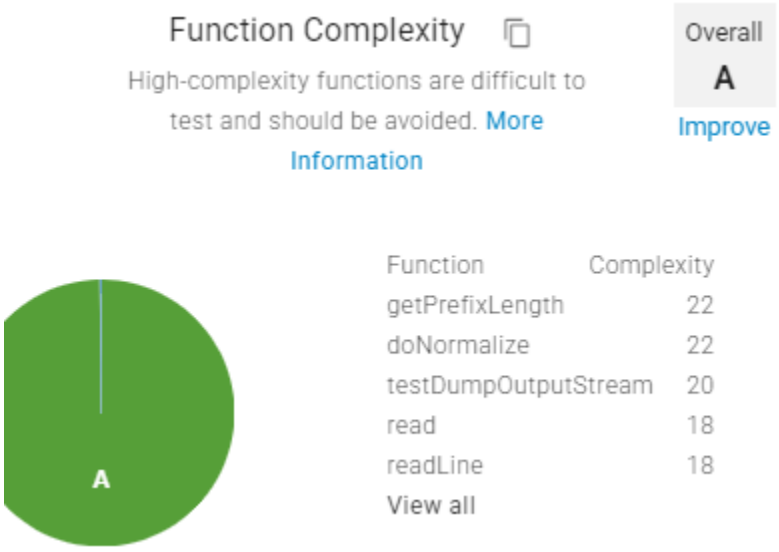
Reason:

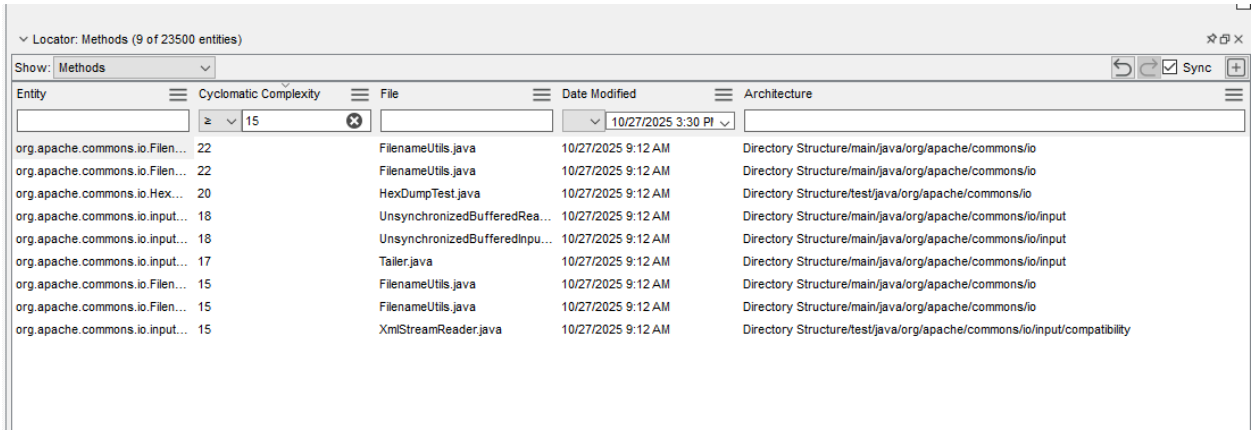
- uses error checks, buffering, and end-of-stream detection to control low-level character reading.
- Character encodings, buffer states, and input validation are handled by several branches.
- Control statements are heavily used to guarantee accurate reading in any situation.

5. **Tailer.java =17 (run):**

Reason:

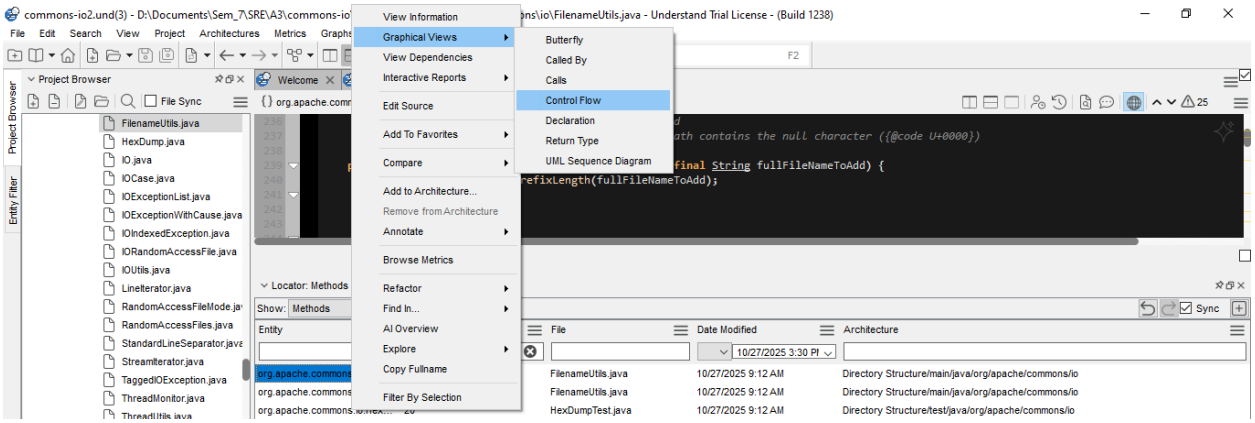
- keeps an eye out for fresh content in a file (similar to the Unix tail -f command).
- includes conditional checks, loops, and exception handling for delays, interruptions, and file reading.
- Concurrent file access and constant monitoring logic lead to complexity.

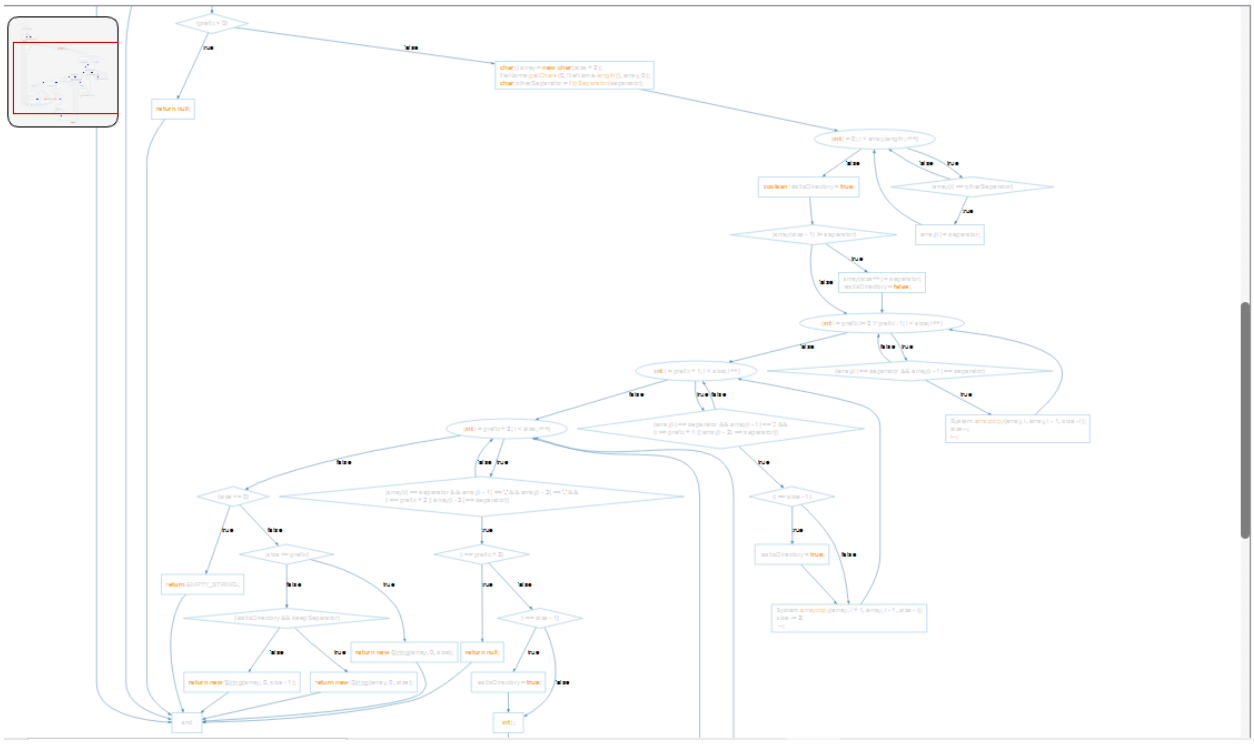




CFG of most complex methods

FilenameUtils.java : 22 (getPrefixLength)





Make Code More Maintainable

The code is hard to read and maintain since the method includes multiple nested `if` blocks that handle various filename patterns (such `~`, drive letters, and file separators).

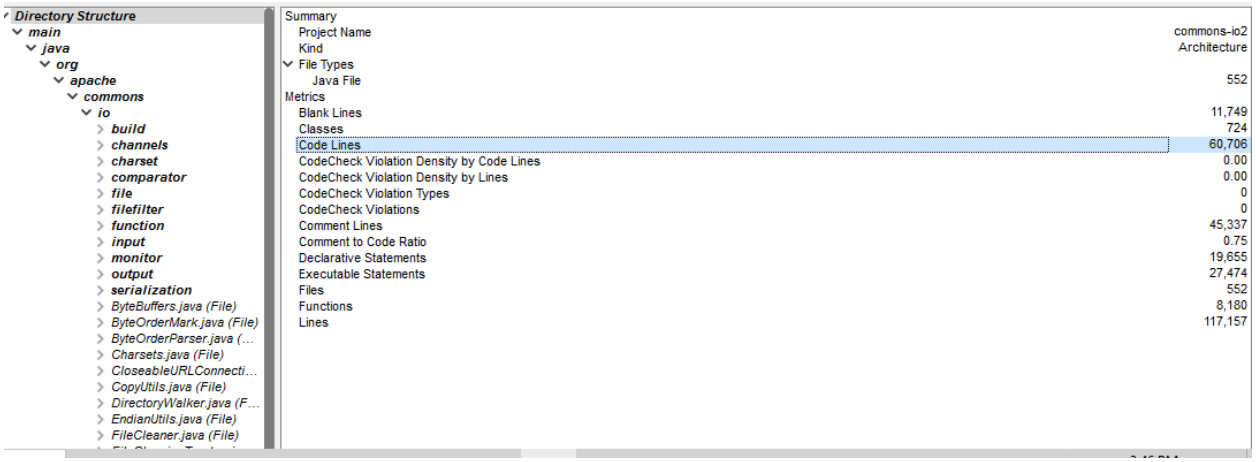
Recommended Enhancement: Divide Logic into Assistive Techniques: Construct a separate private helper method for every conditional situation, such as `handleTildePath()`, `handleDrivePath()`, and `handleSeparatorPath()`. Every assistance should just concentrate on its particular condition or logic for handling patterns.

Advantage: This strategy improves readability, greatly reduces cyclomatic complexity, and makes the process easier to test, debug, and extend in the future.

Code Metrics Evaluation

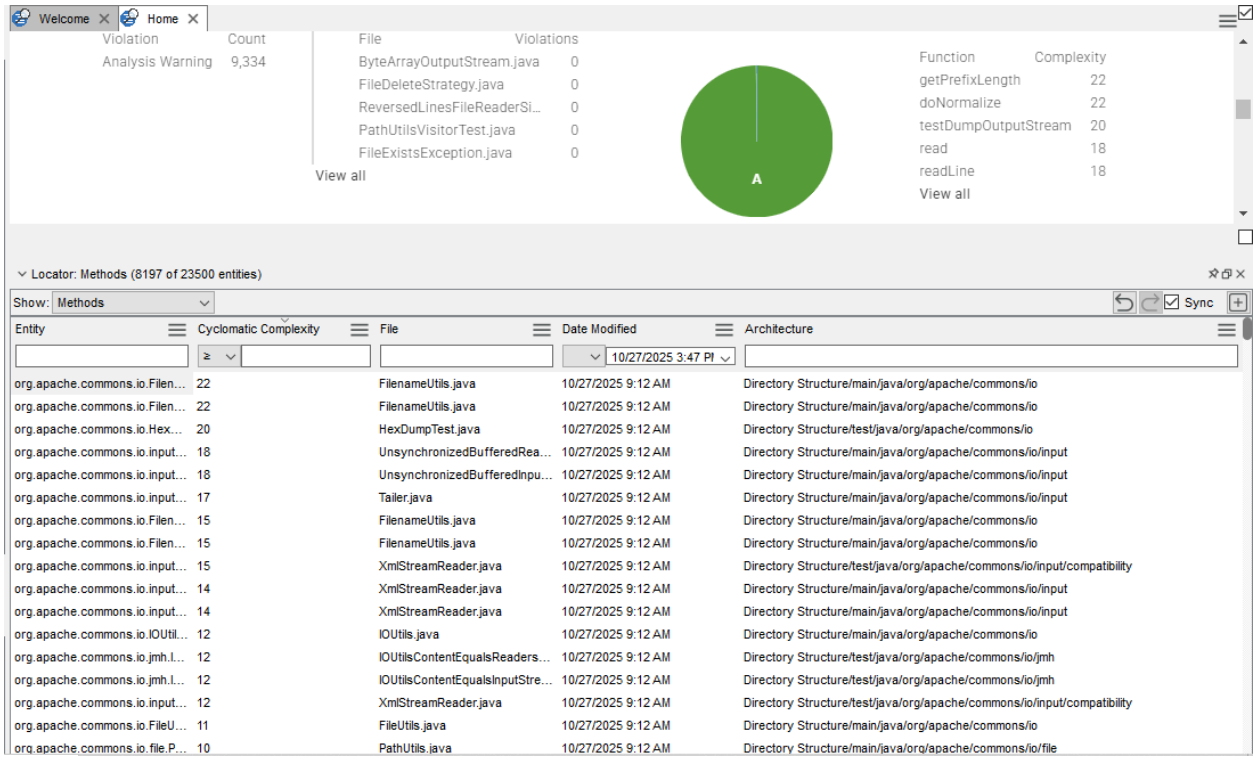
Line of code (LOC)

Total LOC :60705
Average LOC per file: 60705 ÷ 552 = 110 loc



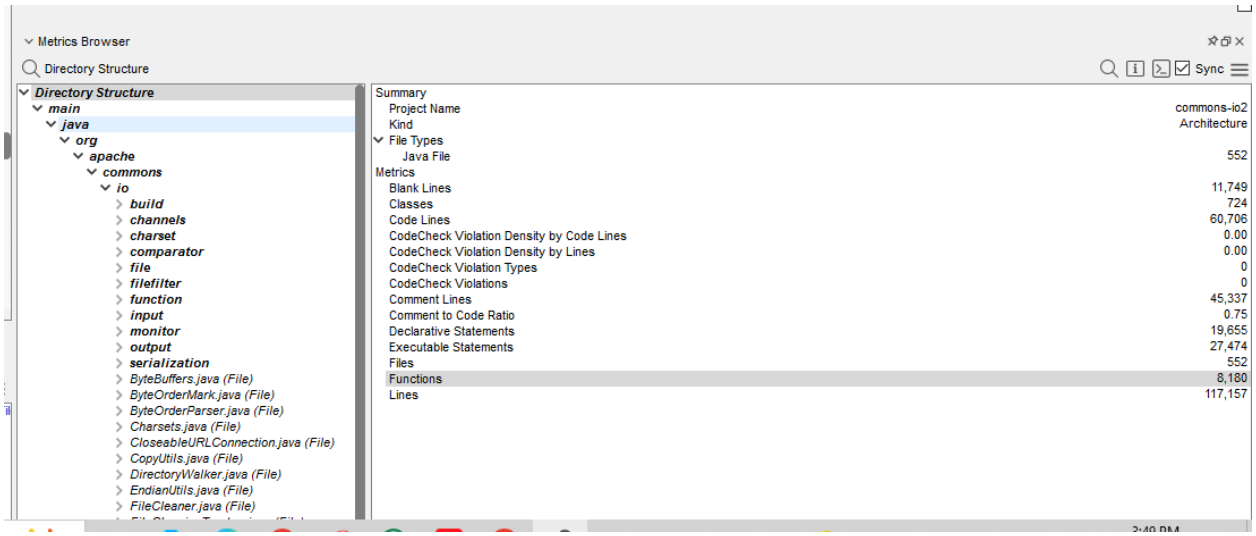
Cyclometric Complexity

We have average Cyclomatic Complexity of 1.27 as shown below :



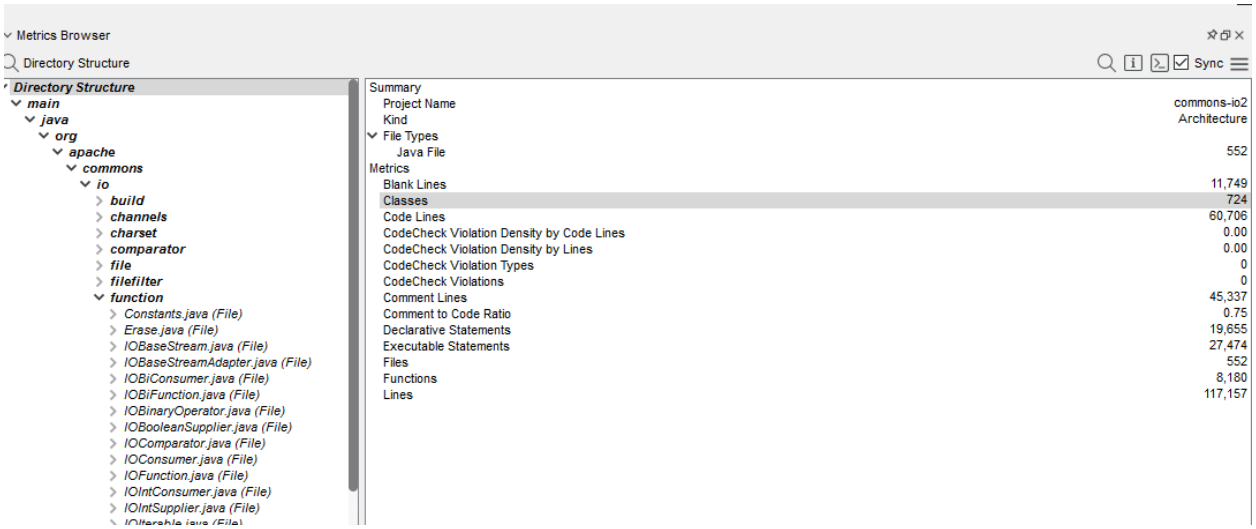
Function Count

There are a total of 8180 functions in our project.



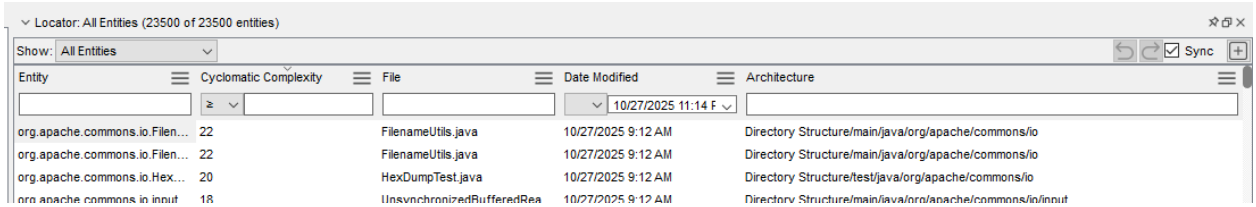
Class Count

There are a total 724 classes in our project.



Maximum Function or Method Complexity

The function with max complexity of 22 is shown as below.



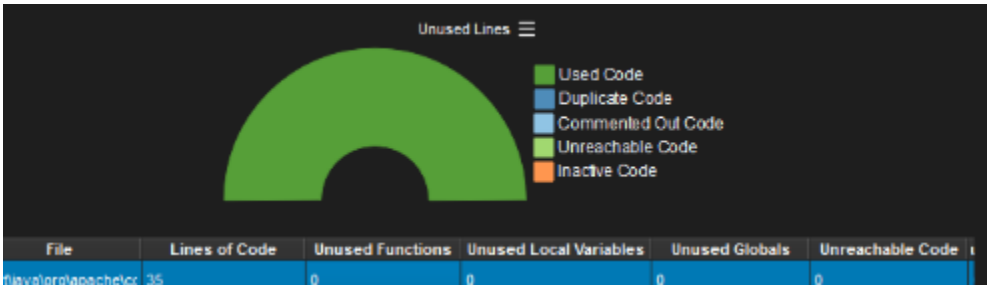
Uninitialized Variables

For this project we notice that there were no Uninitialized Variables in the project all the variables were properly made use of.

File	Lines of Code	Unused Functions	Unused Local Variables	Unused Globals	Unreachable Code
test\java\org\apache\cc	35	0	0	0	0
main\java\org\apache\c	16	0	0	0	0
test\java\org\apache\cc	20	0	0	0	0
test\java\org\apache\cc	34	0	0	0	0
main\java\org\apache\c	104	0	0	0	0
test\java\org\apache\cc	14	0	0	0	0
test\java\org\apache\cc	18	0	0	0	0
test\java\org\apache\cc	49	0	0	0	0

Unreachable Code

For this project we notice that there was no Uninitialized code in the project all the code was properly made use of .



Memory Leaks:

Because of this, memory leaks are not supported for Java programs; only C++ and Python applications are supported.

4. Dependency and Relationship Exploration

Classes Selection

Criteria

- Lines of Code (LOC) ≥ 100
- Number of methods ≥ 5
- Average method cyclomatic complexity ≥ 5

According to the given criteria following 5 classes were selected:

1. org.apache.commons.io.input.wildcardfilefiltest
2. org.apache.commons.io.HexDumpTest
3. org.apache.commons.io.comparator.LastModifiedFileComparatorTest
4. commons\io\comparator\LastModifiedFileComparatorTest.java
5. org.apache.commons.io.file.CountingPathVisitor

Visualizations of selected classes

1. wildcardfilefilertest

- **Butterfly Graph**

Noticeable Dependencies

1. A heavy dependence on JUnit for assertions (many calls to validation).

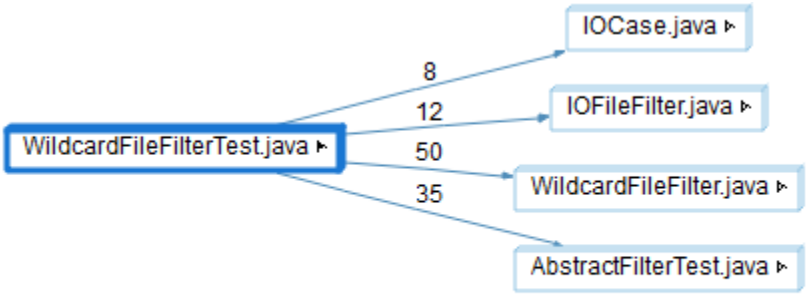
2. Extensive usage of file manipulation classes, such as BoundedReader and TempFile. Intense boundary testing is indicated by frequent readLine and skip calls.

Key Observations

1. Weak modularization due to high connectivity to external utilities and I/O components.

2. Refactoring is required; setup and teardown should be moved to helper methods.

3. To improve test isolation and decrease dependencies, use BoundedReader mocks and stubs.



- **Call Graph:**

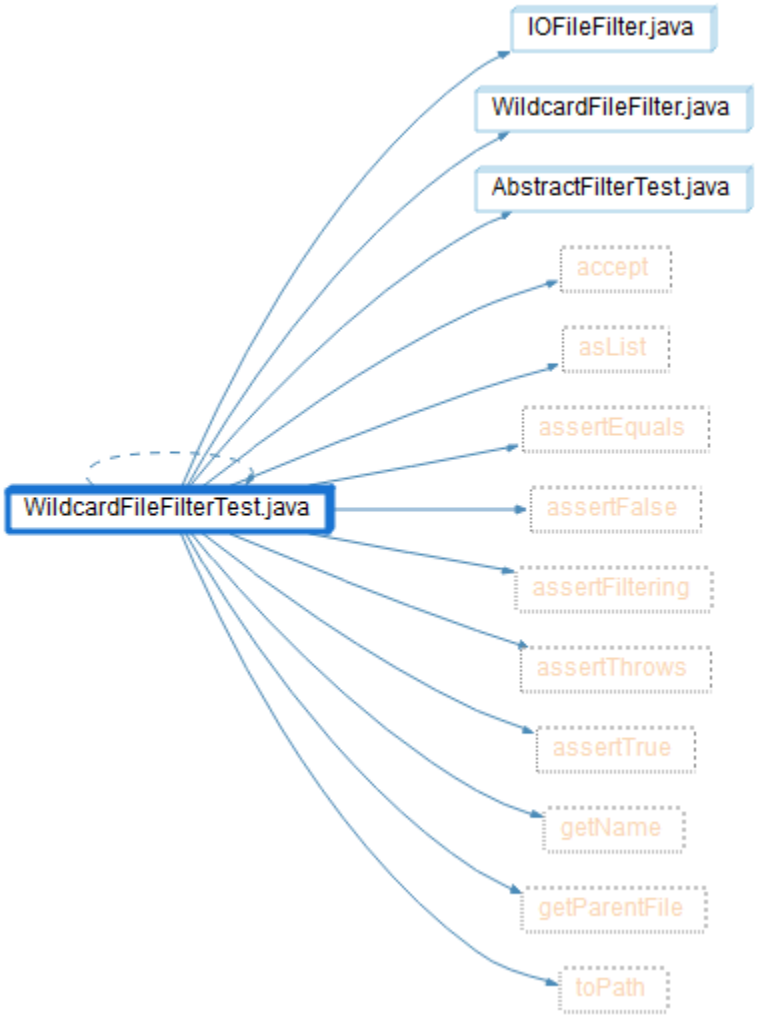
Noticeable Dependencies

1. Strong reliance on assertion techniques and I/O wrappers, similar to the butterfly graph above.

2. Shows that closely connected interactions are visualized repeatedly or redundantly.

Key Observations

1. Consistent call patterns or tool-generated redundancy may cause duplication.
2. Reaffirms worries about limited modularity and high coupling.
3. Refactoring recommendation: to increase modularity and decrease duplication, combine repetitive I/O calls into parameterized tests.



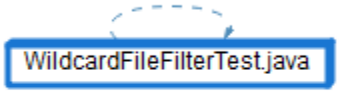
- **Call-by Graph:**

Noticeable Dependencies

1. None visible: a root or solitary view with no connections displayed.

Key Observations

1. None visible: a root or solitary view with no connections displayed. 1. A notion of initial dependency or under-analysis is suggested by minimal representation.
2. While apparent isolation may suggest effective encapsulation, other graphs show actual substantial coupling.
3. Possibility for improvement: avoid deceptive isolation effects by using modular test design or more lucid visualization tools.



- **File Dependency Graph:**

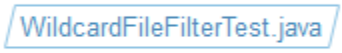
Noticeable Dependencies

1. Robust connections between utility classes and test files.
2. AbstractPathWrapper.java, FileUtil.java, and BoundedReader.java are all closely connected to BoundedReaderTest.java, which serves as a central hub.
3. Shows a strong dependence on shared file utilities.

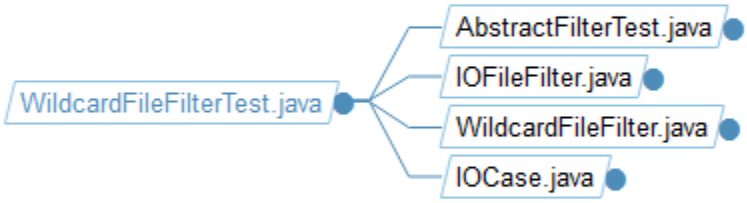
Key Observations

1. Duplication may result from tool-generated redundancy or consistent call patterns.
2. Reaffirms concerns regarding strong coupling and restricted modularity.
3. Refactoring suggestion: include repetitive I/O calls into parameterized tests to improve modularity and reduce duplication.

Depended on by:



Depends on:



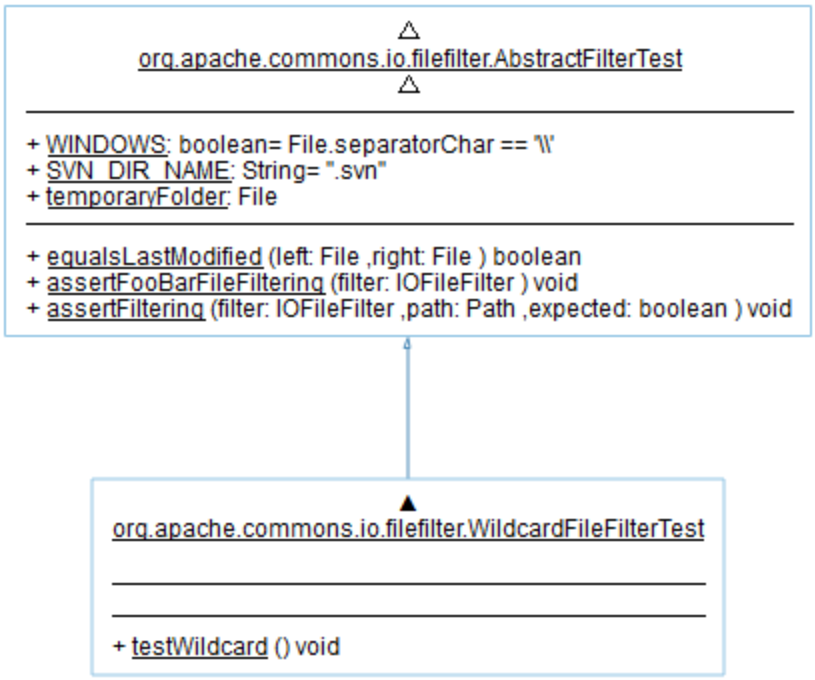
UML Class Diagram:

Noticeable Dependencies

1. Tight coupling to assertion methods, buffered reader, and bounded reader, similar to Graphs 1-2.
2. A monolithic test structure is shown by several consecutive calls (readLine, assertEquals).

Key Observations

1. A lengthy, systematic approach exhibits inadequate modularization.
2. The risk of cascading failures is increased when all scenarios are included in a single test.
3. Refactoring recommendations:
    a. Divide into more manageable, targeted test procedures (one for each boundary scenario).
    b. To improve readability, isolation, and debugging, use JUnit's @ParameterizedTest.



2. org.apache.commons.io.HexDumpTest

- **Butterfly Graph:**

Notable Dependencies:

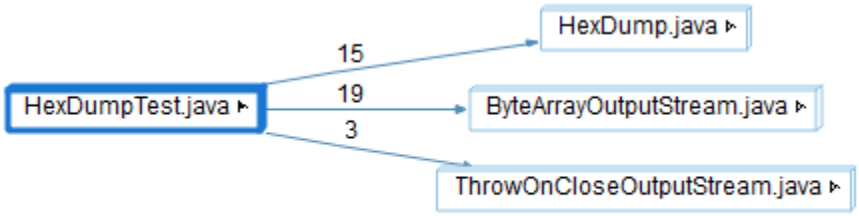
1. Closely related to output stream classes (ThrowOnCloseOutputStream, ByteArrayOutputStream).
2. Hex dumping testing heavily relies on I/O.
3. Extensive validation logic is shown by several assertion calls.

Key Design Observations:

1. A high connection to utilities and I/O indicates mixed concerns and inadequate modularization.
2. Refactoring recommendations:

a. Move dump operations into a utility or assist class.

b. To increase flexibility and decrease direct dependencies, use dependency injection for streams.



- **Call Graph:**

Notable Dependencies:

1. Strong, tightly coupled relationships with multiple path manipulation visitors (AccumulatorPathVisitor, DeletingPathVisitor).
2. Frequent calls to utility methods like Counters and filters indicate repeated counting logic.

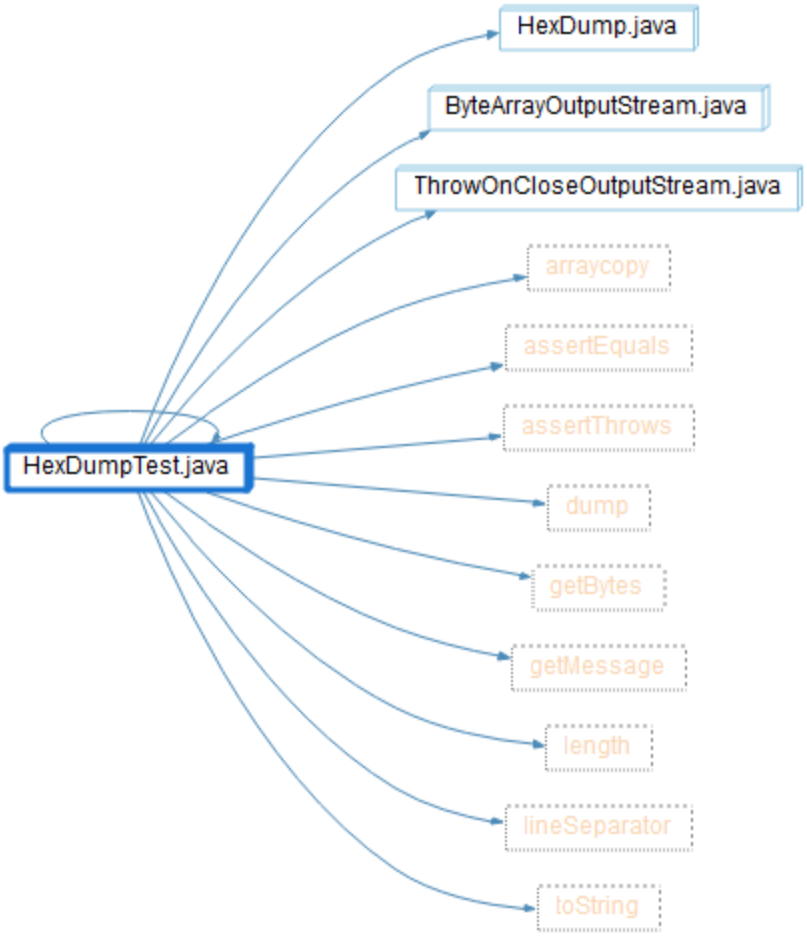
Key Design Observations:

1. Monolithic test architectures are suggested by a consistent strong coupling across perspectives.

2. Refactoring recommendations:

a. Combine repetitive calls using parameterized tests.

b. Reduce duplication to improve maintainability and modularity.



- **Call-By Graph:**

Notable Dependencies:

1. There are no dependencies displayed; this is probably a high-level or unexpanded view devoid of external calls.

Key Design Observations:

1. The high connection observed in other graphs contrasts with the apparent isolation.
2. At this stage, it can suggest either good encapsulation or insufficient analysis.
3. Refactoring recommendations:
    a. Enhance dependency mapping to make visualizations more understandable.

b. No modification is required if isolation is real.
c. Modularize any hidden I/O ties for more streamlined separation.

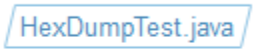
- **File Dependency Graph:**

Notable Dependencies:

1. Implementations of output streams have a strong hierarchical coupling.
2. OutputStream by ByteArray. Java serves as a central hub that is closely connected to proxy and abstract streams.
3. Shows that I/O utilities have too many layers.

Key Design Observations:

1. Dependency on a deep inheritance chain of streams and high file-level coupling.
2. Indicates inadequate modularization and possible design fragility.



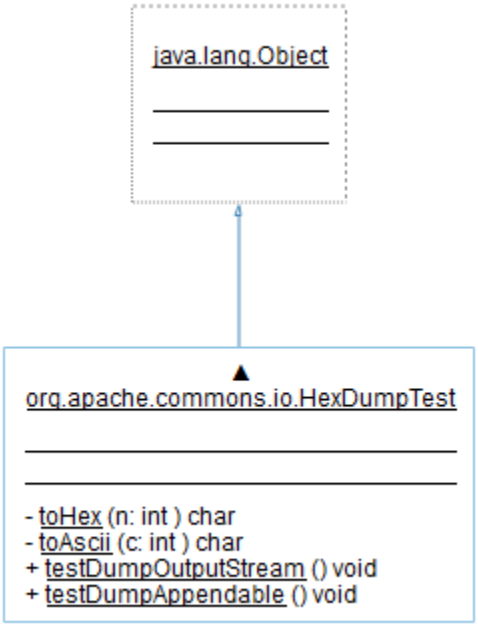
- **UML Class Digram:**

Notable Dependencies:

1. Only the fundamental Object class is extended, with few explicit dependencies.
2. Internal connections to character conversion and dumping logic are implied by method signatures.

Key Design Observations:

1. Only the fundamental Object class is extended, with few explicit dependencies.
2. Internal connections to character conversion and dumping logic are implied by method signatures.



3. commons.io.comparator.LastModifiedFileComparatorTest

- **Butterfly Graph:**

Notable Dependencies:

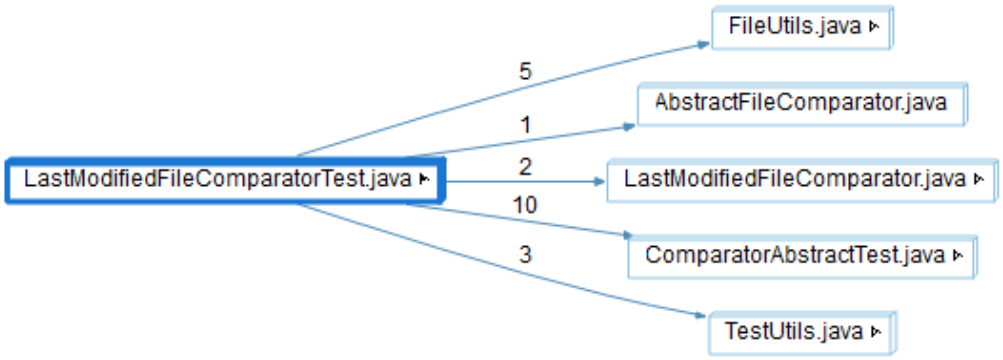
Closely related to techniques for manipulating file timestamps (getLastModified, setLastModified, etc.).

2. Regular usage of path conversion and time-related functions.

Key Design Observations:

1. Closely related to techniques for manipulating file timestamps (getLastModified, setLastModified, etc.).

2. Regular usage of path conversion and time-related functions.



- **Call Graph:**

Notable Dependencies:

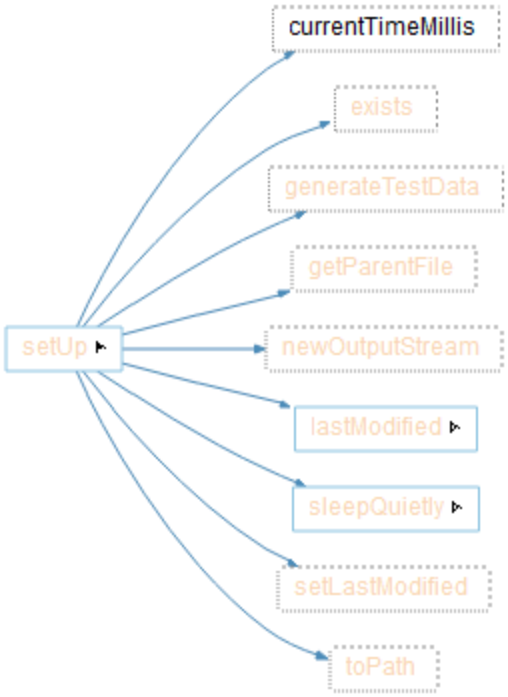
1. Strong correlation between file creation methods and time.

2. Contains calls to lastModified and setLastModified that are chained together.

3. Incorporates sleep functions, implying tests that are time-sensitive.

Key Design Observations:

1. High coupling in setup logic that combines delays, timestamp handling, and file production.
2. Poor modularization and the possibility of faulty tests because of time dependencies are indicated.



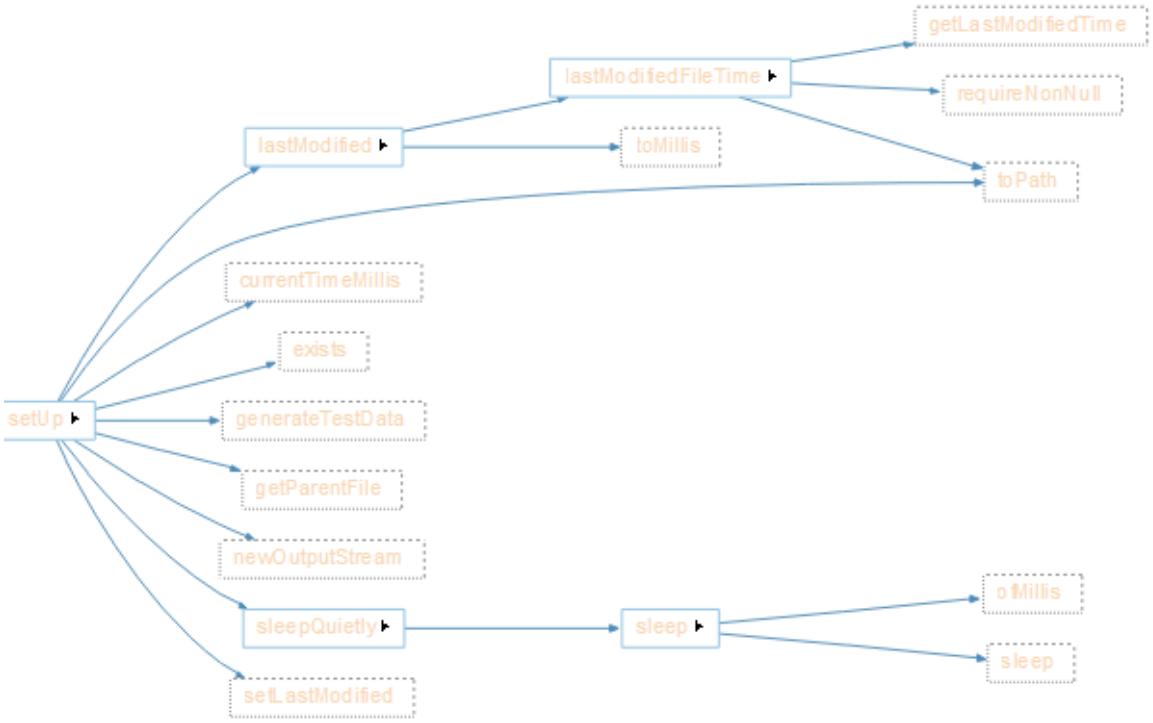
- **Call-By Graph:**

Notable Dependencies:

1. A large volume of calls pouring in from various test situations.
2. In many file activities, timestamp verification is closely linked to lastModified, which serves as a central hub.

Key Design Observations:

1. An excessive dependence on a single validation technique is revealed by high coupling.
2. Shows a lack of test isolation and inadequate modularization.



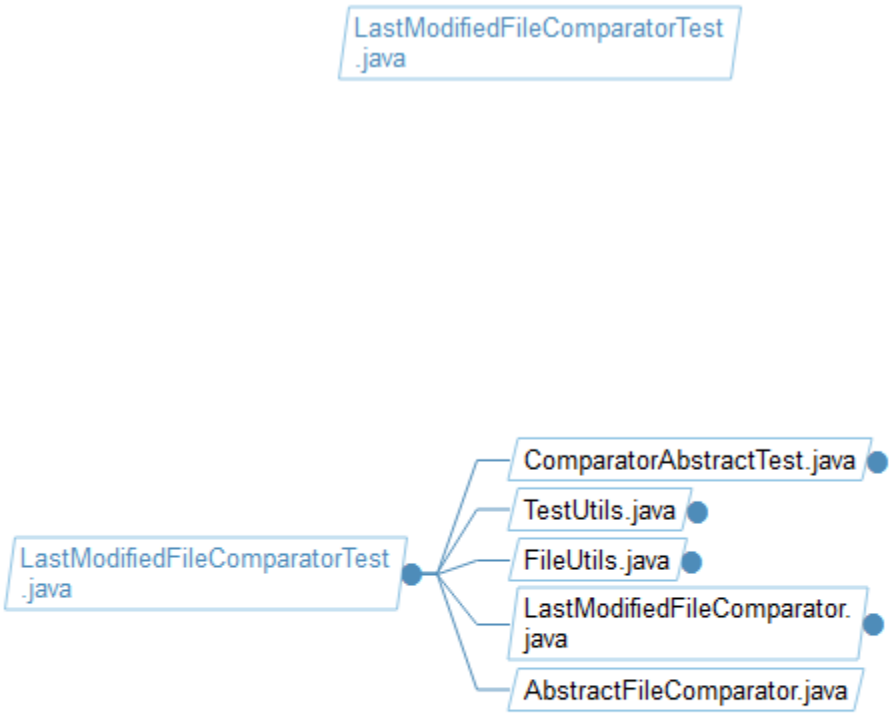
- **File Dependency Graph:**

Notable Dependencies:

1. Strong reliance on abstract comparator classes and close file-level connection.
2. A deep inheritance hierarchy is formed by the test file's heavy reliance on utility files such as FileUtils.java.

Key Design Observations:

1. A lengthy comparator inheritance chain results in poor modularization.
2. If base classes change, tests run the risk of becoming brittle.



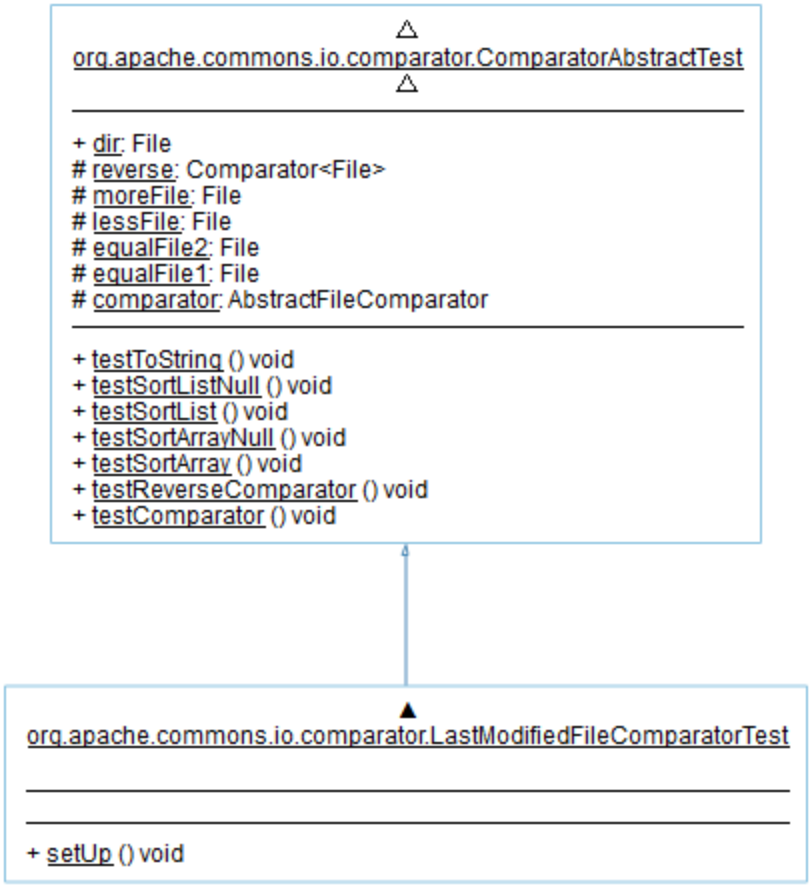
- **UML Class Digram:**

Notable Dependencies:

1. A dependency on the abstract comparator based on inheritance from the test class.
2. A number of sort-related techniques exhibit a close relationship with comparison logic.

Key Design Observations:

1. The boundaries between production and test code are blurred and significant coupling is created when an abstract class is extended in testing.



4. commons\io\comparator\LastModifiedFileComparatorTest.java

- **Butterfly Graph:**

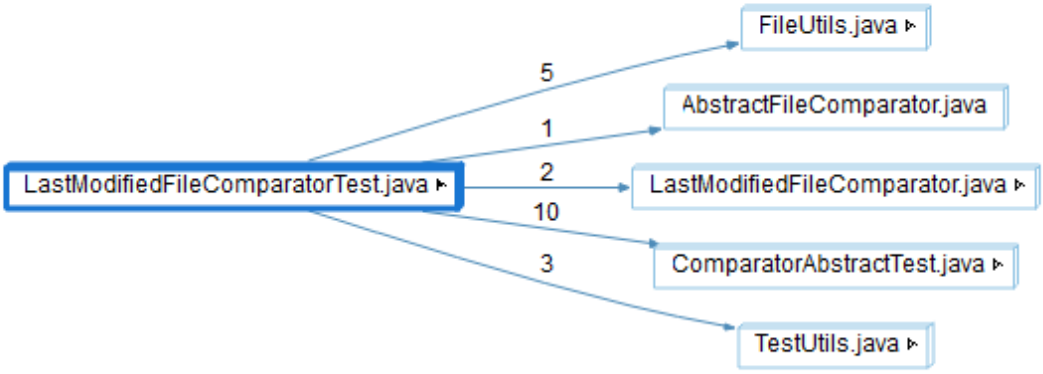
Notable Dependencies:

1. Approximately ten calls to LastModifiedFileComparator.java are excessive.
2. Exhibits close integration with file manipulation tools and fundamental comparator logic.
3. Involves a lot of utility methods in the tests.

Key Design Observations:

1. Poor modularization is revealed by high coupling and weighted dependencies.

2. The test class lacks flexibility due to its excessive reliance on implementation specifics.



- **Call Graph:**

Notable Dependencies:

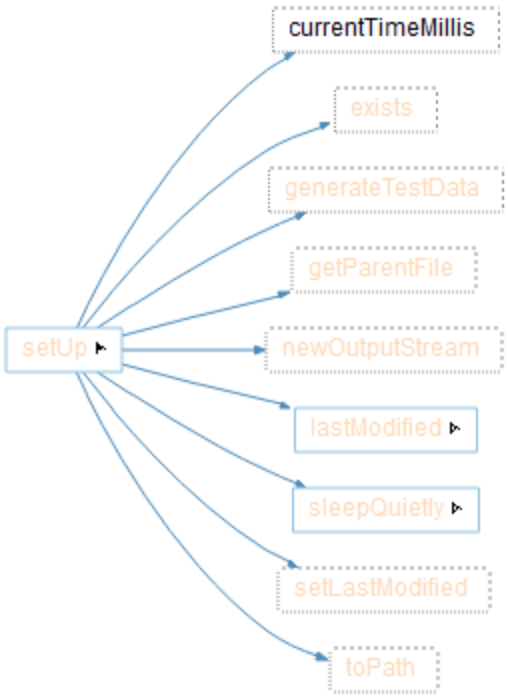
1. Closely related to I/O and timestamp operations.

2. A significant dependence on external file system operations is seen in the numerous chained calls to lastModified and setLastModified.

Key Design Observations:

1. Closely related to I/O and timestamp operations.

2. A significant dependence on external file system operations is seen in the numerous chained calls to lastModified and setLastModified.



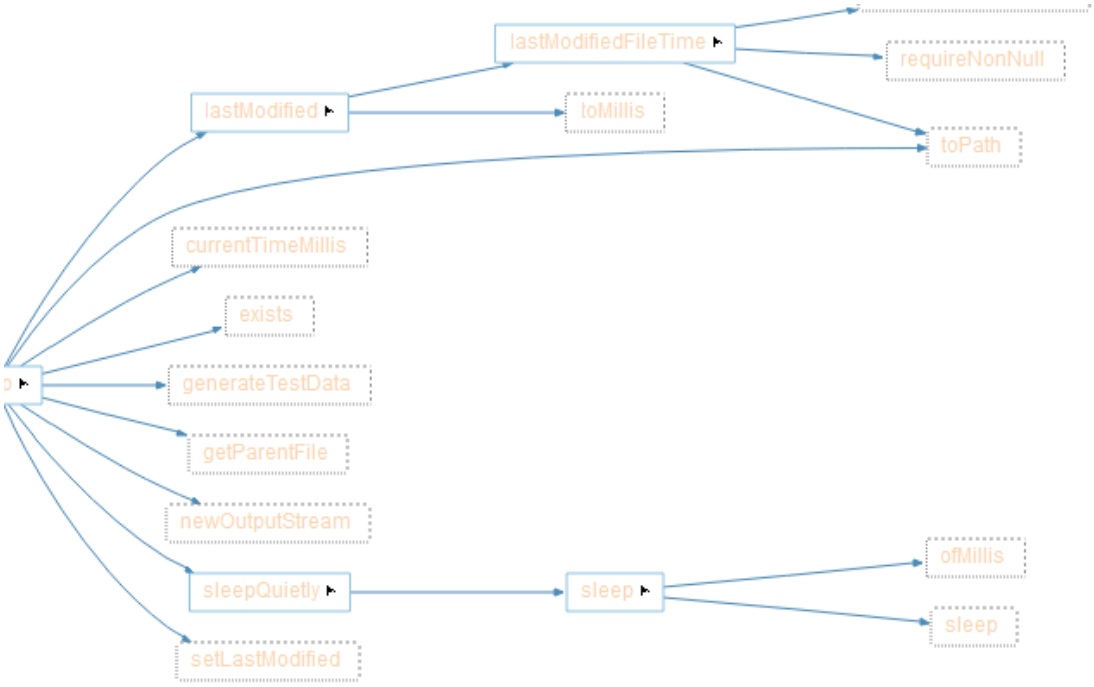
- **Call-By Graph:**

Notable Dependencies:

- Although context from other graphs suggests hidden strong couplings to comparator hierarchies and utilities, none are publicly displayed.

Key Design Observations:

1. In contrast to more general system dependencies, the apparent isolation suggests either effective encapsulation or insufficient visualization at this resolution.



- **File Dependency Graph:**

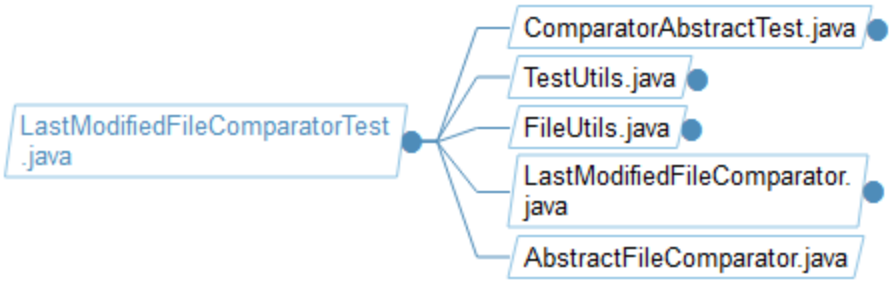
Notable Dependencies:

1. There are no obvious dependencies.

2. Hidden couplings to utility classes and comparator hierarchies are suggested by context from other graphs.

Key Design Observations:

1. Incomplete visualization or strong encapsulation may be indicated by seeming isolation contrasted with known larger dependencies.



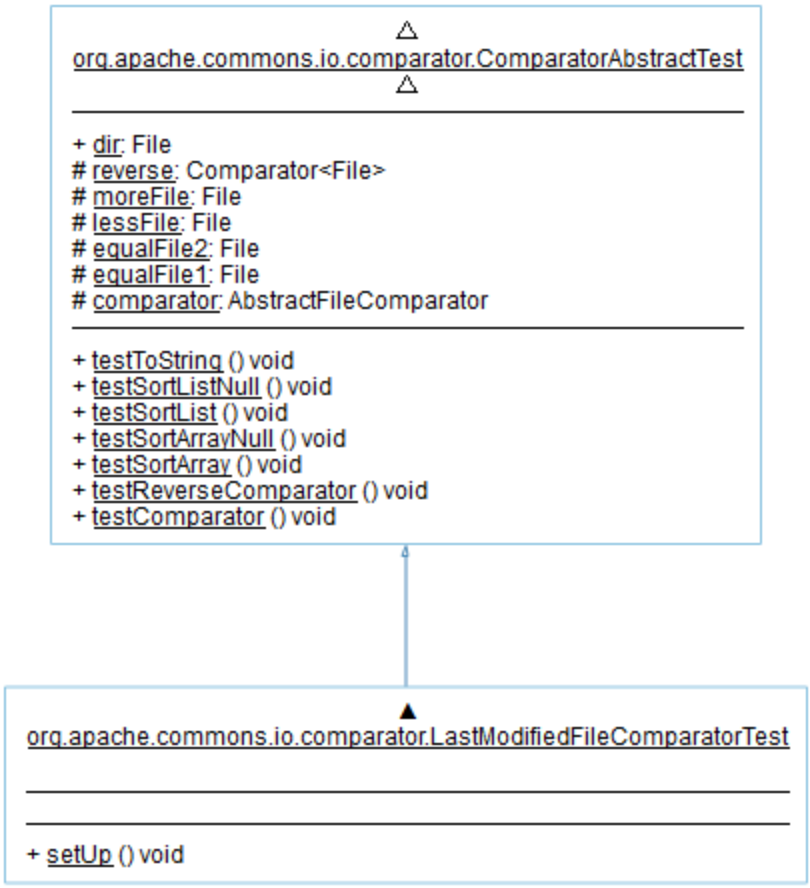
- **UML Class Digram:**

Notable Dependencies:

1. Strong linkage to numerous utility and comparator files with extreme fan-out.
2. shows that there are too many links throughout the file comparator ecosystem.

Key Design Observations:

1. Poor package-level modularization and extremely high coupling.
2. One file acts as an all-powerful center, posing threats to scalability and maintenance.



5. org.apache.commons.io.file.CountingPathVisitor

- **Butterfly Graph:**

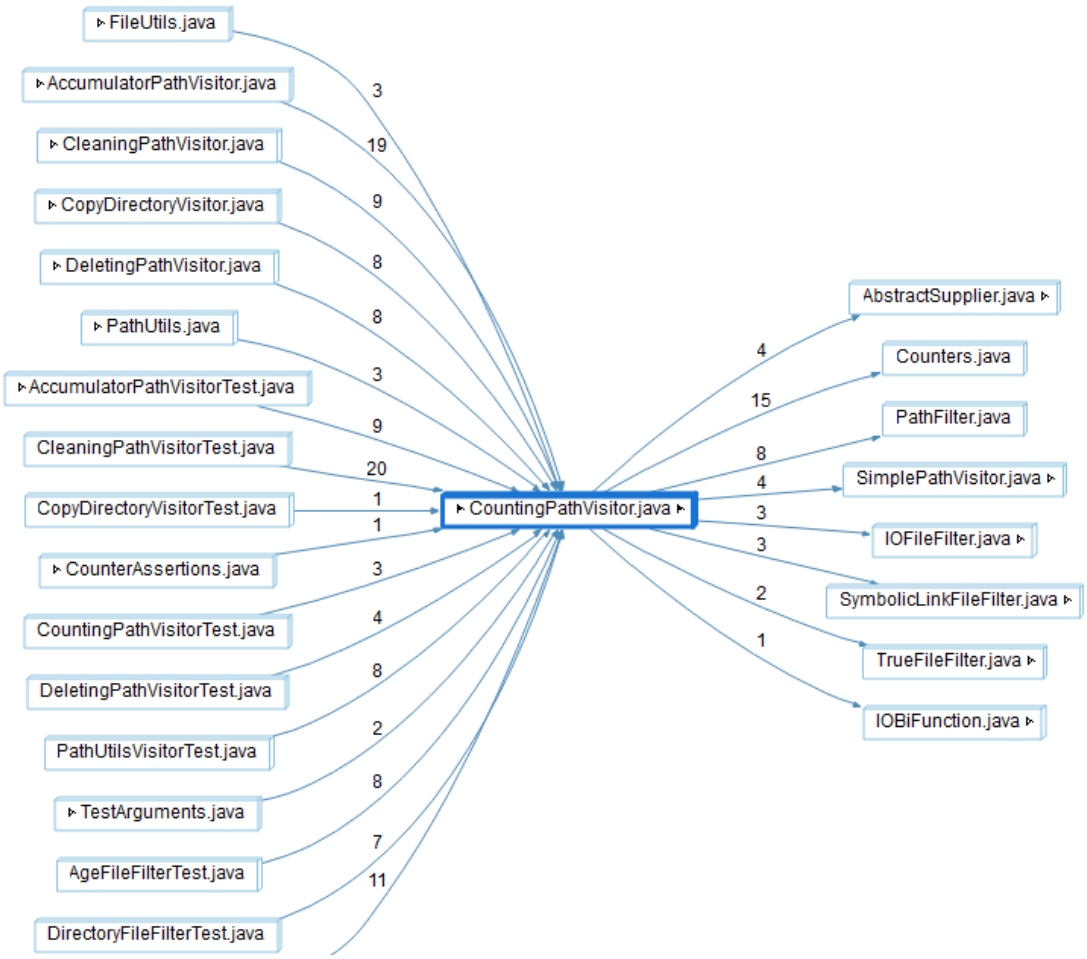
Notable Dependencies:

1. Incoming calls from numerous visitor test classes are closely connected.
2. A lot of file operations rely on CountingPathVisitor for validation.
3. Dense dependency clusters are produced by outgoing dependencies to counter and filter utilities.

Key Design Observations:

1. Poor modularization is indicated by high coupling and centralized logic.

2. CountingPathVisitor risks complexity and rigidity as an overloaded core component.



- **Call Graph:**

Notable Dependencies:

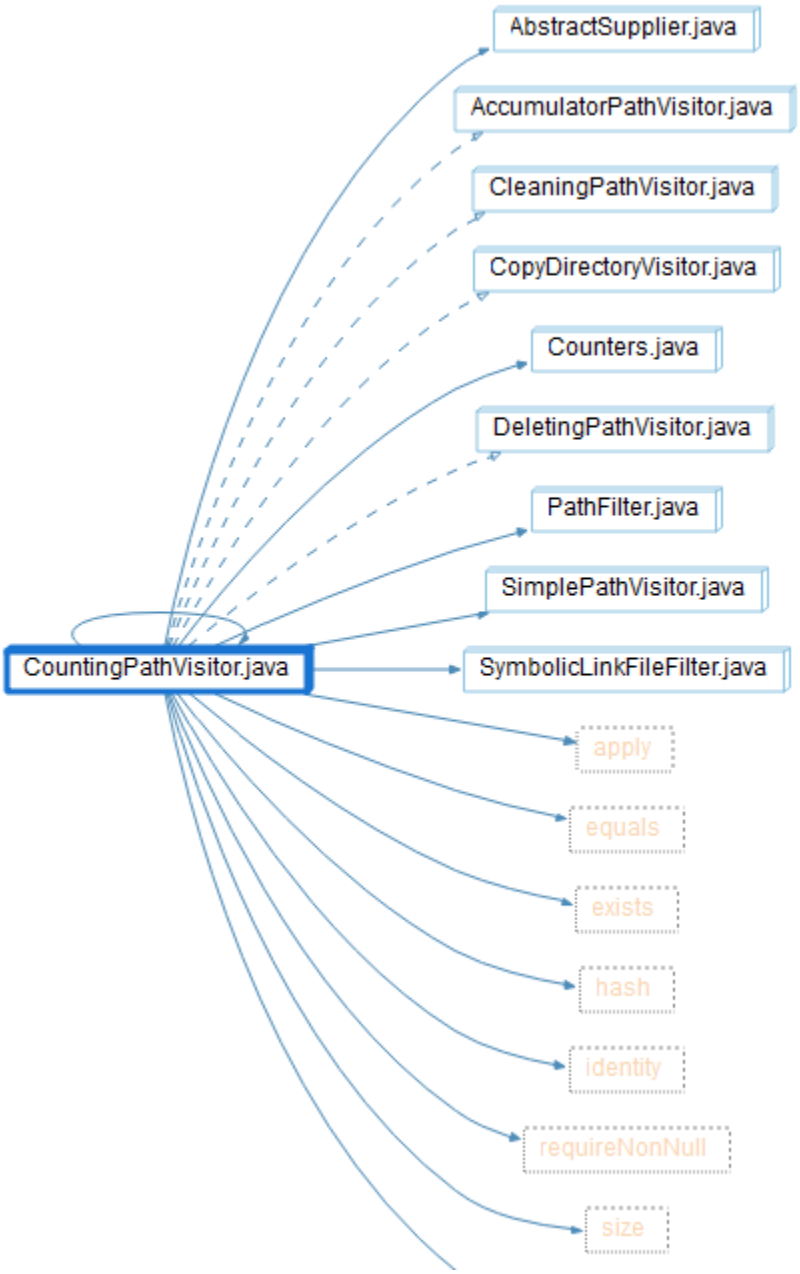
1. Strong, close-knit connections with several path manipulation visitors (DeletingPathVisitor, AccumulatorPathVisitor).

2. Repeated counting logic is shown by frequent calls to utility methods such as counters and filters.

Key Design Observations:

1. Poor modularization is demonstrated by high coupling between visitor classes.

2. Cleaning, removing, and filtering issues are mixed together with counting logic.



- **Call-By Graph:**

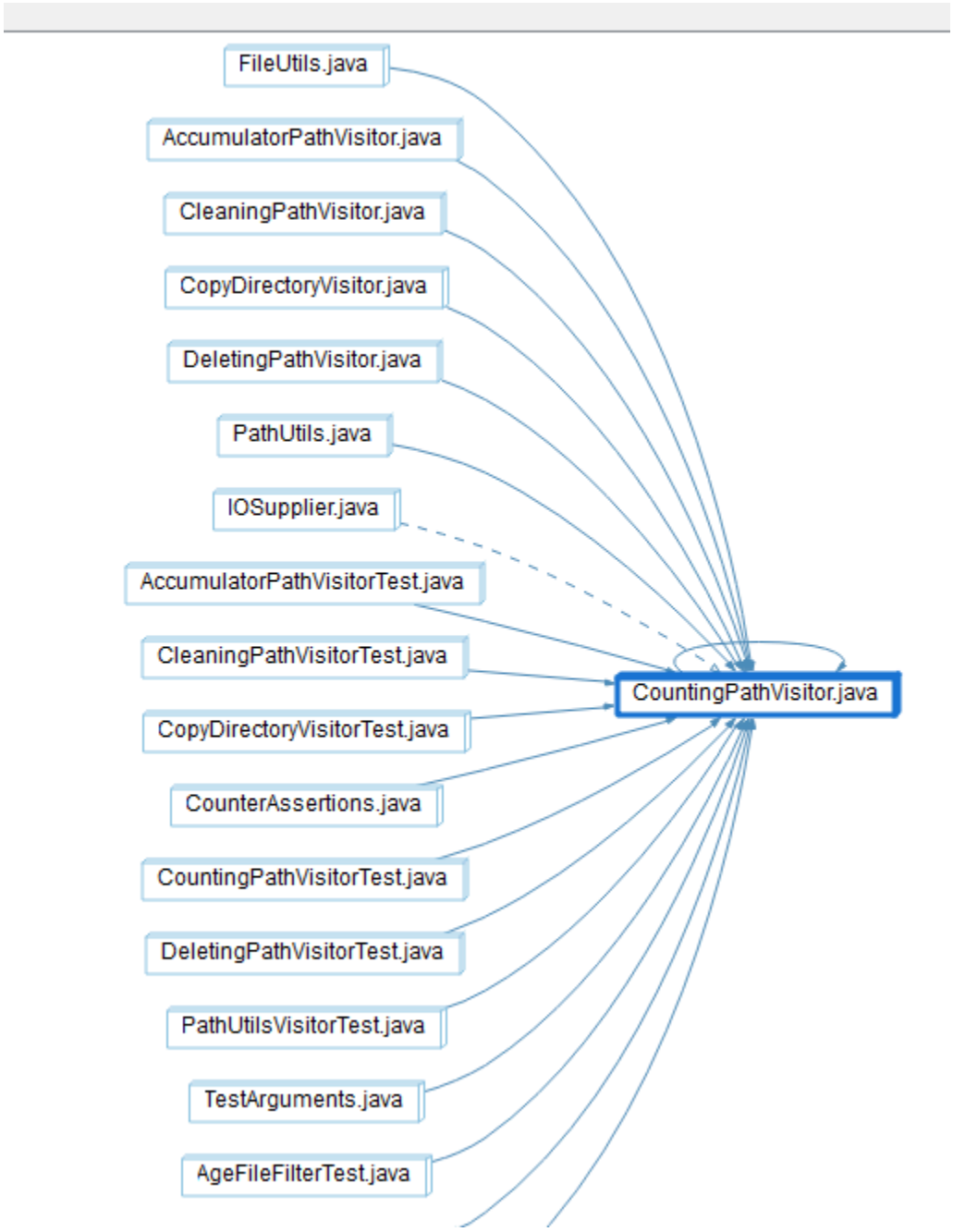
Notable Dependencies:

1. Too many calls coming in from different test classes.
2. For path-related test cases, CountingPathVisitor acts as a tightly connected hub.

Key Design Observations:

1. Over-centralization of logic inside tests is demonstrated by high coupling.

2. Raises the possibility that even little modifications could have a significant effect.



- **File Dependency Graph:**

Notable Dependencies:

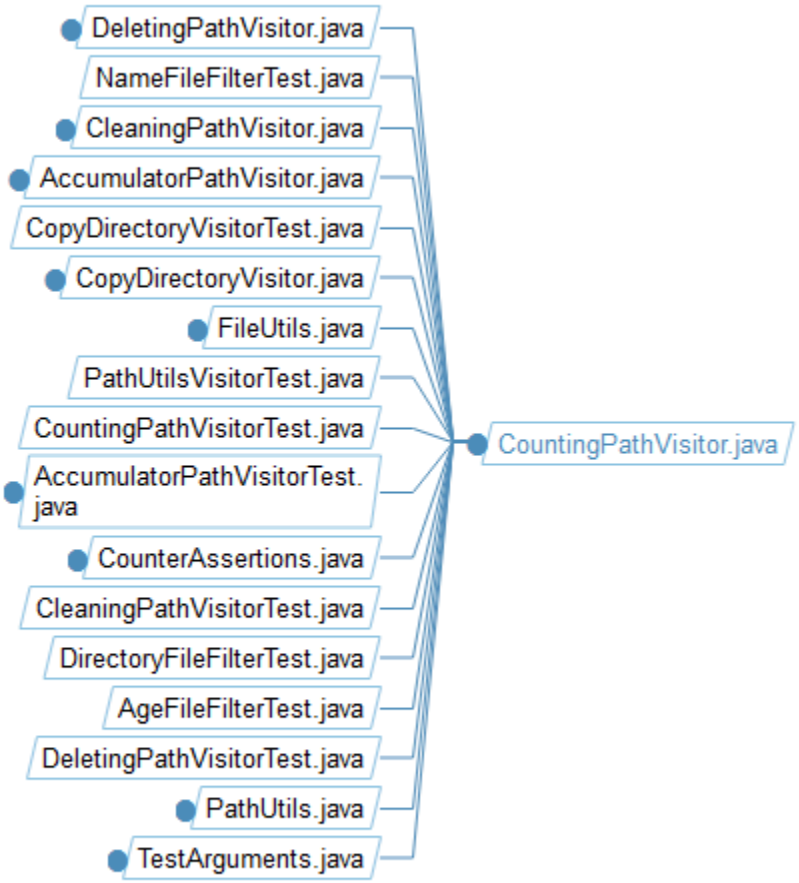
1. Wide-ranging, closely related connections to filter and exception handling classes.

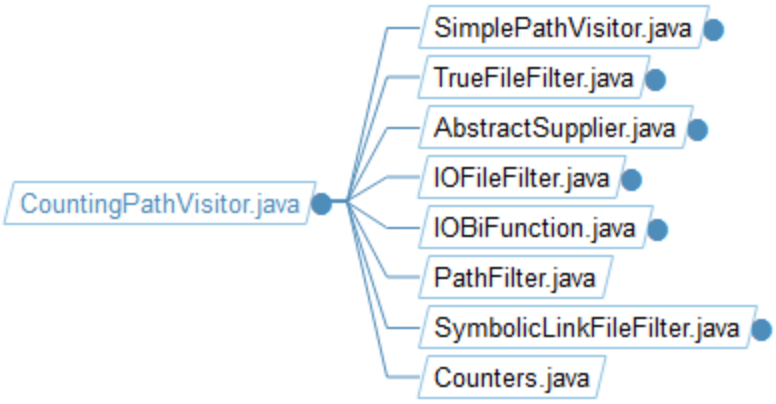
2. Excessive file-level linkages are revealed by deep branching into I/O utilities.

Key Design Observations:

1. The file structure is poorly modularized; one file serves as a hub for all dependencies.

2. Complexity of maintenance and risk compilation cascades.





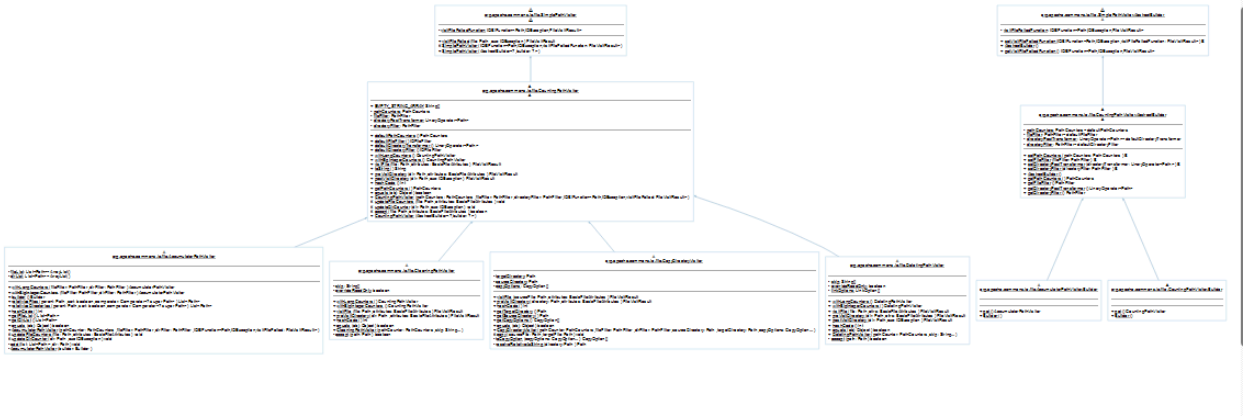
- **UML Class Digram:**

Notable Dependencies:

1. Dependencies on composition and inheritance with abstract counters and visitors.
2. Tight coupling via integrated filters and method overrides.

Key Design Observations:

1. High coupling and possible rigidity in design are indicated by deep inheritance hierarchies.
2. If methods are too specialized for counting tasks, there is poor modularization.



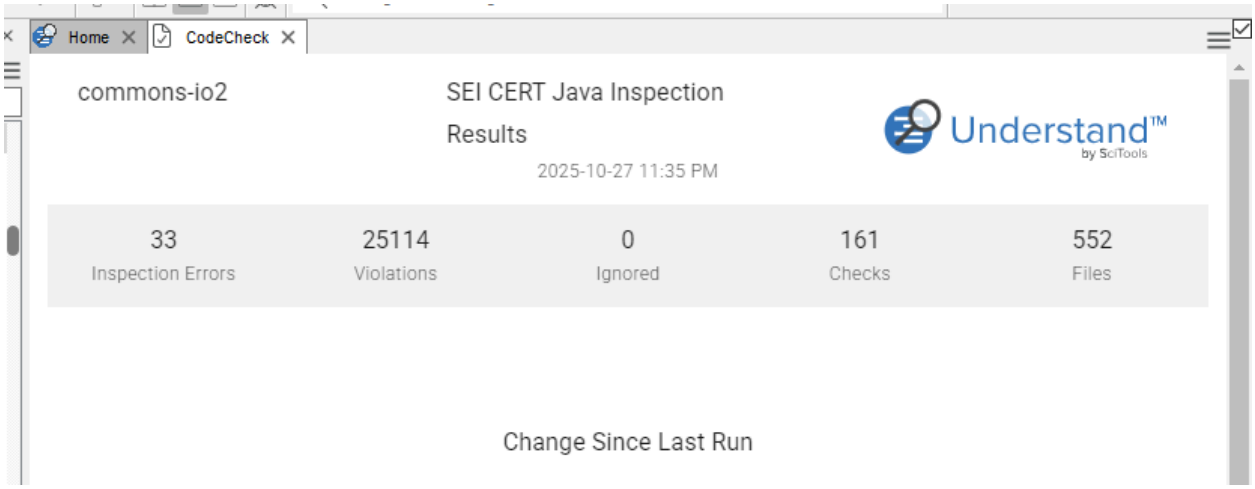
5. Code Check and Violations

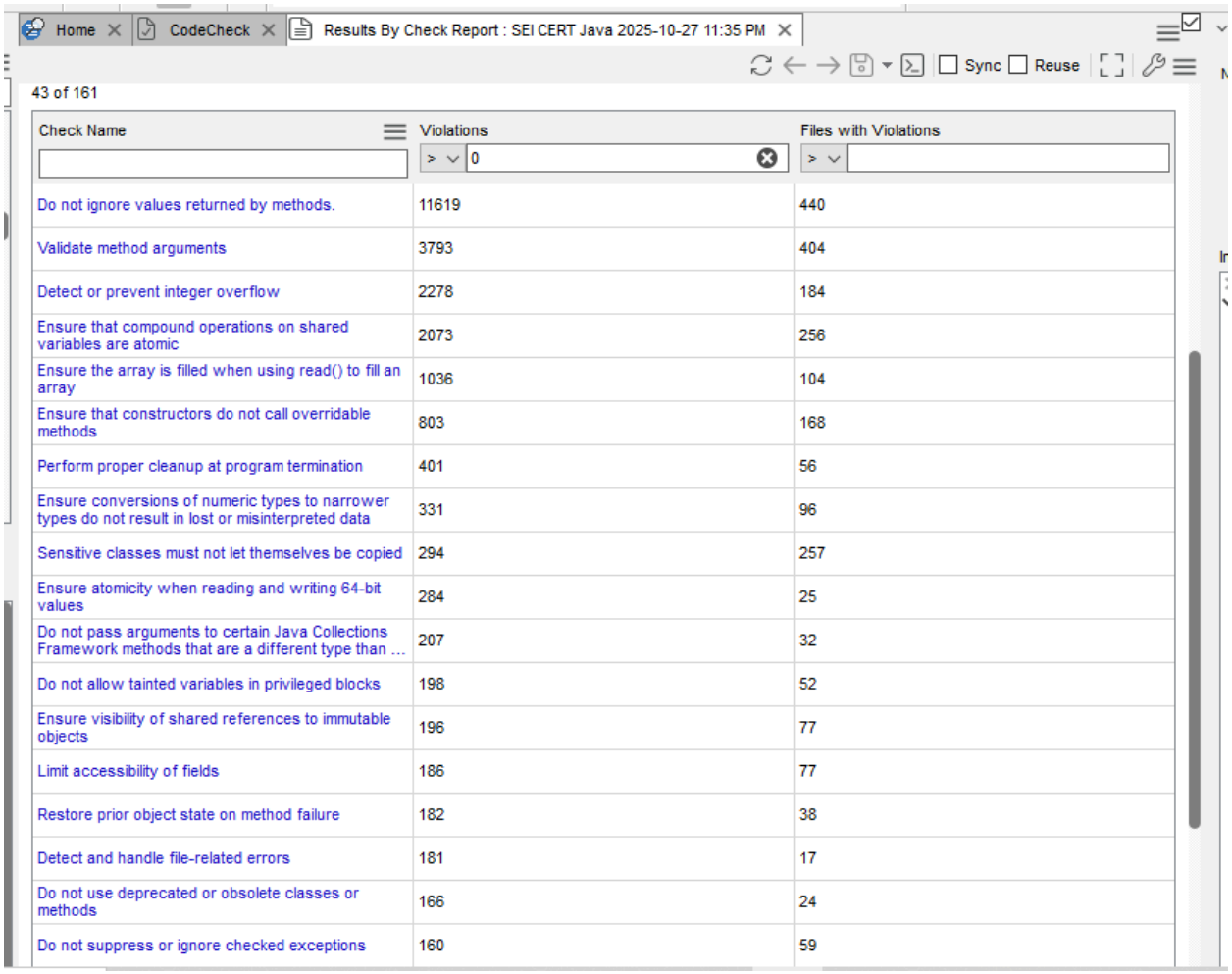
Overview

By spotting unnecessary code, logical problems, and violations of coding standards, code checking guarantees program quality and consistency. It enhances the codebase's readability, maintainability, and dependability.

Code Analysis

The entire project was scanned for standard violations, unnecessary imports, and naming discrepancies using SciTools Understand's Code Check Tool. To identify areas that required improvement, identified problems were grouped, examined, and exported into a report.





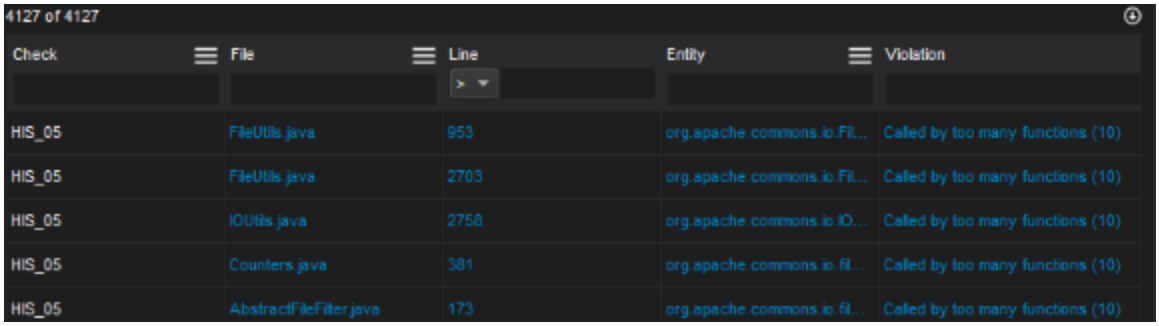
Check Name	Violations	Files with Violations
	> 0	>
Do not ignore values returned by methods.	11619	440
Validate method arguments	3793	404
Detect or prevent integer overflow	2278	184
Ensure that compound operations on shared variables are atomic	2073	256
Ensure the array is filled when using read() to fill an array	1036	104
Ensure that constructors do not call overridable methods	803	168
Perform proper cleanup at program termination	401	56
Ensure conversions of numeric types to narrower types do not result in lost or misinterpreted data	331	96
Sensitive classes must not let themselves be copied	294	257
Ensure atomicity when reading and writing 64-bit values	284	25
Do not pass arguments to certain Java Collections Framework methods that are a different type than ...	207	32
Do not allow tainted variables in privileged blocks	198	52
Ensure visibility of shared references to immutable objects	196	77
Limit accessibility of fields	186	77
Restore prior object state on method failure	182	38
Detect and handle file-related errors	181	17
Do not use deprecated or obsolete classes or methods	166	24
Do not suppress or ignore checked exceptions	160	59

Coding Standards Violation

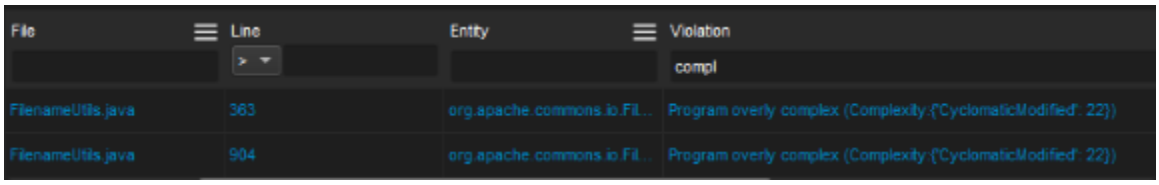
High VOCF: Several classes, including EndianUtils.java, FileUtils.java, and FilenameUtils.java, have high VOCF (complexity metric) violations (numbers ranging from 4.02 to 17.26).

Check	File	Line	Entity	Violation
		>		
HIS_11	FilenameUtilsTest.java	102	org.apache.commons.io.Fil...	VOCF too high(9.96)
HIS_11	FileUtilsTest.java	634	org.apache.commons.io.Fil...	VOCF too high(9.82)
HIS_11	XmlStreamReader.java	783	org.apache.commons.io.in....	VOCF too high(9.80)
HIS_11	XmlStreamReader.java	597	org.apache.commons.io.in....	VOCF too high(9.69)
HIS_11	RandomAccessFilesTest.java	51	org.apache.commons.io.R...	VOCF too high(9.58)

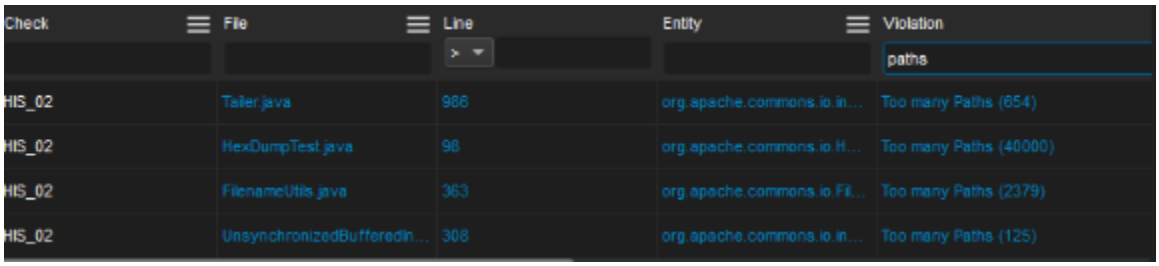
Excessive call levels: in methods like DirectoryWalker.walk (up to 10 levels).



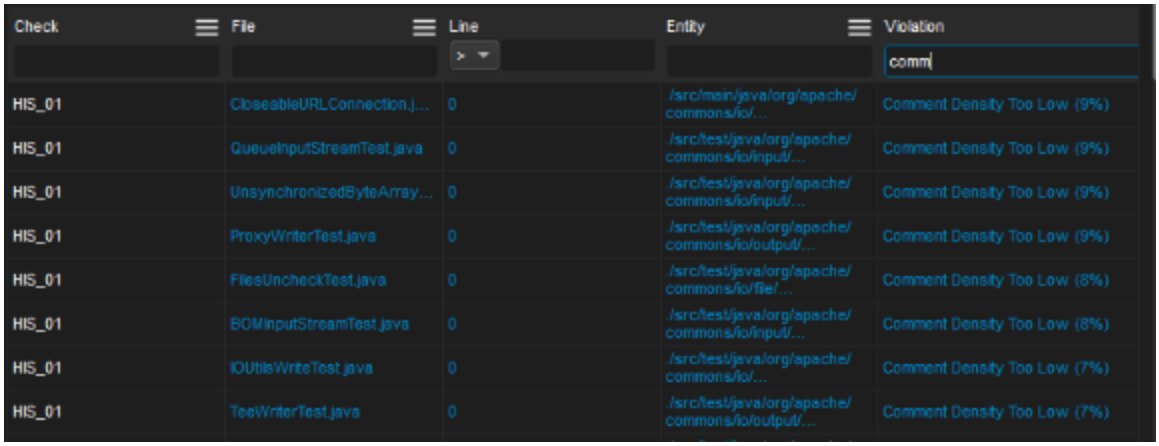
Overly complex programs: with high CyclomaticModified complexity (e.g., 22 in FilenameUtils.doNormalize).



Excessive paths: in methods like FilenameUtils.doNormalize (2379 paths) and Tailer.run (654 paths).



Low comment density: (below 20%) in numerous test and source files like CloseableURLConnection.java (9%) and BOMInputStreamTest.java (8%).



Check	File	Line	Entity	Violation
		>		comm
HIS_01	CloseableURLConnection.j...	0	/src/main/java/org/apache/commons/io/...	Comment Density Too Low (9%)
HIS_01	QueueInputStreamTest.java	0	/src/test/java/org/apache/commons/io/input/...	Comment Density Too Low (9%)
HIS_01	UnsynchronizedByteArray...	0	/src/test/java/org/apache/commons/io/input/...	Comment Density Too Low (9%)
HIS_01	ProxyWriterTest.java	0	/src/test/java/org/apache/commons/io/output/...	Comment Density Too Low (9%)
HIS_01	FilesUncheckTest.java	0	/src/test/java/org/apache/commons/io/file/...	Comment Density Too Low (8%)
HIS_01	BOMInputStreamTest.java	0	/src/test/java/org/apache/commons/io/input/...	Comment Density Too Low (8%)
HIS_01	IOUtilsWriteTest.java	0	/src/test/java/org/apache/commons/io/...	Comment Density Too Low (7%)
HIS_01	TeeWriterTest.java	0	/src/test/java/org/apache/commons/io/output/...	Comment Density Too Low (7%)

Logical or Structural Warnings

Techniques like Uncheck.apply (invoked by up to 34 functions) and IOSupplier.get (invoked by 14 functions) show high fan-in, which indicates that they are overused and could become challenging to safely adjust.

●FileUtils.deleteDirectory and other functions with deep call chains (up to 11 levels) expose structural complexity and complicate logic tracing or debugging.

Due to high cyclomatic and path metrics, a number of algorithms exhibit logical complexity, indicating a higher likelihood of untested or error-prone execution paths

●If any dependant component changes, methods with many dependencies, like ProxyInputStream.read (used by 17 functions), present possible reliability and maintainability concerns.

Code Violation Report

Over 300 problems were found in the test and source files, mostly in the org.apache.commons.io package. For reference, the entire set of results has been exported to a CSV file.

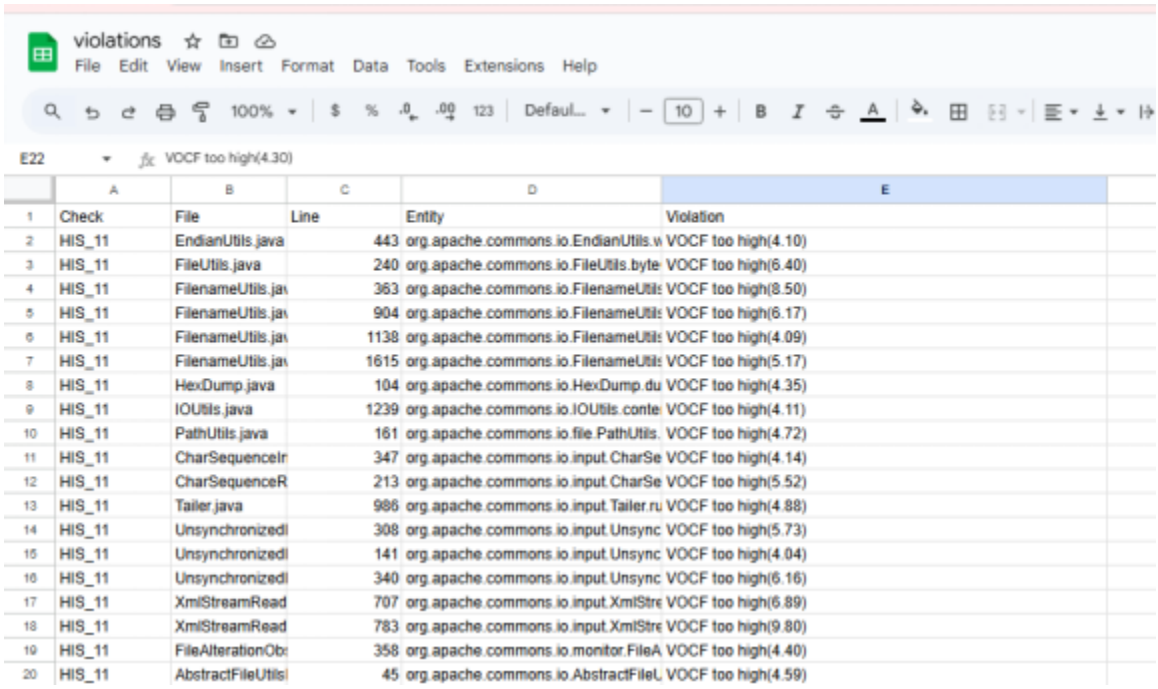
Key Files Impacted:

- FilenameUtils.java is one of the key files affected; it displays several high VOCF and cyclomatic complexity ratings.
- The test cases in FileUtilsTest.java have high VOCF scores.
- IOUtilsTest.java has multiple infractions because of its high VOCF.
- XmlStreamReader.java has deep call level problems and a high VOCF.

Violation Summary by rule:

- HIS_11: More than 200 infractions for abnormally high VOCF (e.g., test methods exceeding 20.49).
- HIS_09: Over 50 cases of excessive call depth, including file deletion methods with up to 11 nested layers.
- Around 200+ instances of methods being called by too many functions (such as 77 calls to the ByteArrayOutputStream constructor) are reported in HIS_05.
- HIS_02: More than five infractions for excessive pathways, including 40,000 paths found in HexDumpTest.
- HIS_04: Two techniques were deemed excessively complicated, with cyclomatic complexity rising to 22.
- HIS_01: Over 100 instances of low comment density (in test and source files, only 7–19%).

Overall, the analysis shows that a number of core and test files have inadequate documentation, high levels of structural and logical complexity, and unnecessary dependencies.


violations

File Edit View Insert Format Data Tools Extensions Help

E22 VOCF too high(4.30)

	A	B	C	D	E
1	Check	File	Line	Entity	Violation
2	HIS_11	EndianUtils.java	443	org.apache.commons.io.EndianUtils.w	VOCF too high(4.10)
3	HIS_11	FileUtils.java	240	org.apache.commons.io.FileUtils.byte	VOCF too high(6.40)
4	HIS_11	FilenameUtils.jav	363	org.apache.commons.io.FilenameUtils	VOCF too high(8.50)
5	HIS_11	FilenameUtils.jav	904	org.apache.commons.io.FilenameUtils	VOCF too high(6.17)
6	HIS_11	FilenameUtils.jav	1138	org.apache.commons.io.FilenameUtils	VOCF too high(4.09)
7	HIS_11	FilenameUtils.jav	1615	org.apache.commons.io.FilenameUtils	VOCF too high(5.17)
8	HIS_11	HexDump.java	104	org.apache.commons.io.HexDump.du	VOCF too high(4.35)
9	HIS_11	IOUtils.java	1239	org.apache.commons.io.IOUtils.conte	VOCF too high(4.11)
10	HIS_11	PathUtils.java	161	org.apache.commons.io.file.PathUtils.	VOCF too high(4.72)
11	HIS_11	CharSequenceIr	347	org.apache.commons.io.input.CharSe	VOCF too high(4.14)
12	HIS_11	CharSequenceR	213	org.apache.commons.io.input.CharSe	VOCF too high(5.52)
13	HIS_11	Tailer.java	986	org.apache.commons.io.input.Tailer.ru	VOCF too high(4.88)
14	HIS_11	UnsynchronizedI	308	org.apache.commons.io.input.Unsync	VOCF too high(5.73)
15	HIS_11	UnsynchronizedI	141	org.apache.commons.io.input.Unsync	VOCF too high(4.04)
16	HIS_11	UnsynchronizedI	340	org.apache.commons.io.input.Unsync	VOCF too high(6.16)
17	HIS_11	XmlStreamRead	707	org.apache.commons.io.input.XmlStre	VOCF too high(6.89)
18	HIS_11	XmlStreamRead	783	org.apache.commons.io.input.XmlStre	VOCF too high(9.80)
19	HIS_11	FileAlterationOb	358	org.apache.commons.io.monitor.FileA	VOCF too high(4.40)
20	HIS_11	AbstractFileUtils	45	org.apache.commons.io.AbstractFileU	VOCF too high(4.59)

Summary of issues and impact

Aspect	Details
Types of Issues	The codebase shows high complexity across multiple metrics (VOCF, number of paths, and Cyclomatic Complexity). There are also deep call hierarchies and high fan-in/fan-out values. However, no issues were found related to unused imports or naming conventions.
Impact on Reliability	Elevated complexity and numerous execution paths can lead to hidden bugs and untested scenarios, decreasing the system's reliability. Deep call stacks also introduce potential stack overflow risks in recursive or heavily nested methods.
Impact on Maintainability	Methods with many dependencies and deep call structures make refactoring and debugging more challenging. Additionally, low comment density hinders developers from understanding or safely modifying existing logic.
Impact on Readability	The scarcity of comments (7–19%) reduces code clarity and makes it difficult to understand the purpose and flow of complex sections, especially within test files. Intricate control structures further slow down code comprehension.

Key Violation and brief interpretation

Methods having high levels of control flow, decisions, or pathways run the danger of untested scenarios, defects, and poor reliability; they are also more difficult to maintain or refactor. This is known as high complexity (HIS_11, HIS_04, and HIS_02).

```
    private int testFile2Size;

    @BeforeEach
    public void setUp() throws Exception {
        testFile1 = Files.createTempFile(temporaryFolder, "test", "1");
        testFile2 = Files.createTempFile(temporaryFolder, "test", "2");

        testFile1Size = (int) Files.size(testFile1);
        testFile2Size = (int) Files.size(testFile2);
        if (!Files.exists(testFile1.getParent())) {
            throw new IOException("Cannot create file " + testFile1
                    + " as the parent directory does not exist");
        }
        try (BufferedOutputStream output3 =
                new BufferedOutputStream(Files.newOutputStream(testFile1))) {
            TestUtils.generateTestData(output3, testFile1Size);
        }
        if (!Files.exists(testFile2.getParent())) {
            throw new IOException("Cannot create file " + testFile2
                    + " as the parent directory does not exist");
        }
        try (BufferedOutputStream output2 =
                new BufferedOutputStream(Files.newOutputStream(testFile2))) {
            TestUtils.generateTestData(output2, testFile2Size);
        }
        if (!Files.exists(testFile1.getParent())) {
            throw new IOException("Cannot create file " + testFile1
```

Excessive call nesting in Deep Structures (HIS_09) can result in stack overflows, make debugging more difficult, and lower performance and maintainability.

```
public static String byteCountToDisplaySize(final BigInteger size) {
    Objects.requireNonNull(size, "size");
    final String displaySize;
    if (size.divide(ONE_QB).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_QB) + " QB";
    } else if (size.divide(ONE_RB).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_RB) + " RB";
    } else if (size.divide(ONE_YB).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_YB) + " YB";
    } else if (size.divide(ONE_ZB).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_ZB) + " ZB";
    } else if (size.divide(ONE_EB_BI).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_EB_BI) + " EB";
    } else if (size.divide(ONE_PB_BI).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_PB_BI) + " PB";
    } else if (size.divide(ONE_TB_BI).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_TB_BI) + " TB";
    } else if (size.divide(ONE_GB_BI).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_GB_BI) + " GB";
    } else if (size.divide(ONE_MB_BI).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_MB_BI) + " MB";
    } else if (size.divide(ONE_KB_BI).compareTo(BigInteger.ZERO) > 0) {
        displaySize = size.divide(ONE_KB_BI) + " KB";
```

High Dependencies (HIS_05): Excessive reusing of methods damages maintainability and scalability by increasing the impact of changes.

```java
package org.apache.commons.io;

import java.io.Serializable;
import java.nio.charset.StandardCharsets;
import java.util.Locale;
import java.util.Objects;
```

```java
public String getCharsetName() {
    return charsetName;
}

int[] getRawBytes() {
    return bytes;
}

/**
 * Computes the hash code for this BOM.
 *
 * @return the hash code for this BOM.
 * @see Object#hashCode()
 */
@Override
public int hashCode() {
    int hashCode = getClass().hashCode();
    for (final int b : bytes) {
        hashCode += b;
    }
    return hashCode;
}

/**
 * Gets the length of the BOM's bytes.
 *
 * @return the length of the BOM's bytes
 */
public int length() {
    return bytes.length;
}
```

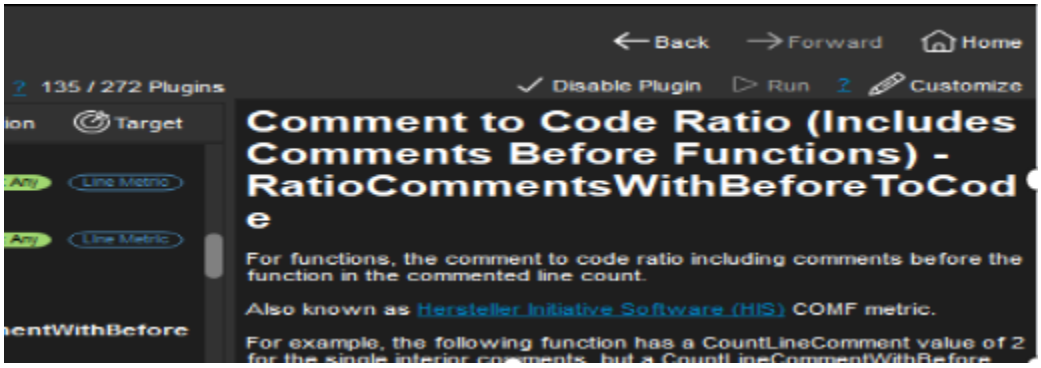
Bad Documentation (HIS_01): Code that has few comments is difficult for teams to understand or onboard, which affects maintainability over the long run.

```
@Override
public boolean getDoInput() {
    return urlConnection.getDoInput();
}

@Override
public boolean getDoOutput() {
    return urlConnection.getDoOutput();
}

@Override
public long getExpiration() {
    return urlConnection.getExpiration();
}

@Override
public String getHeaderField(final int n) {
    return urlConnection.getHeaderField(n);
}

@Override
public String getHeaderField(final String name) {
    return urlConnection.getHeaderField(name);
```

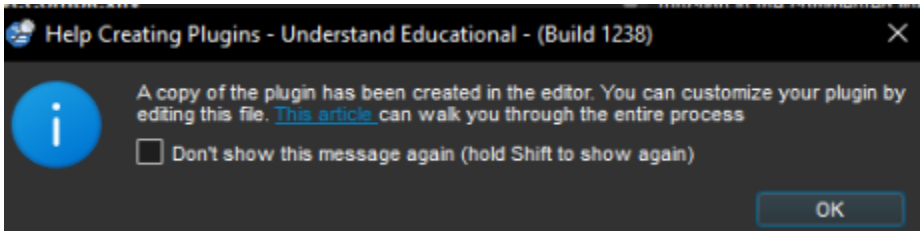
6. Advanced Querying

Comment-to-Code Ratio Analysis (Top five files)

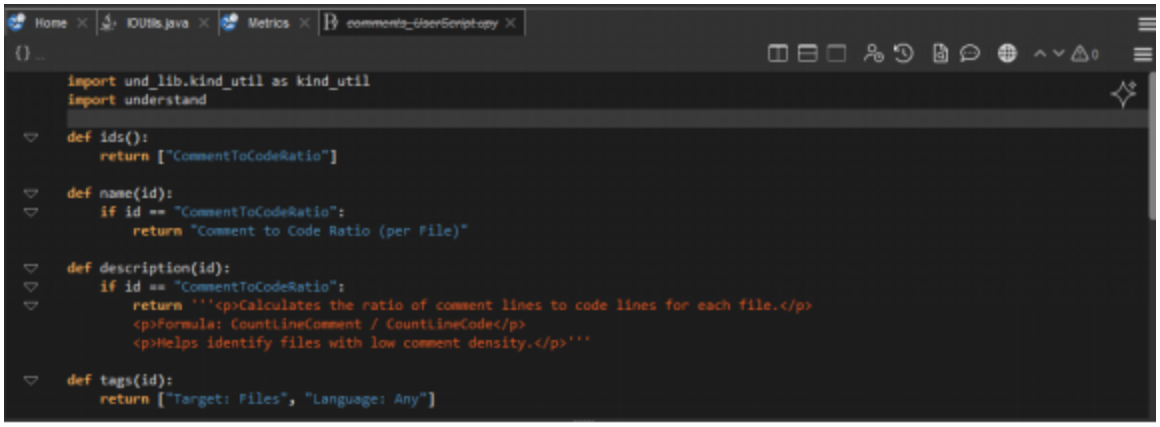
1. Customizing the comment to code ratio plugin metric is the first step.



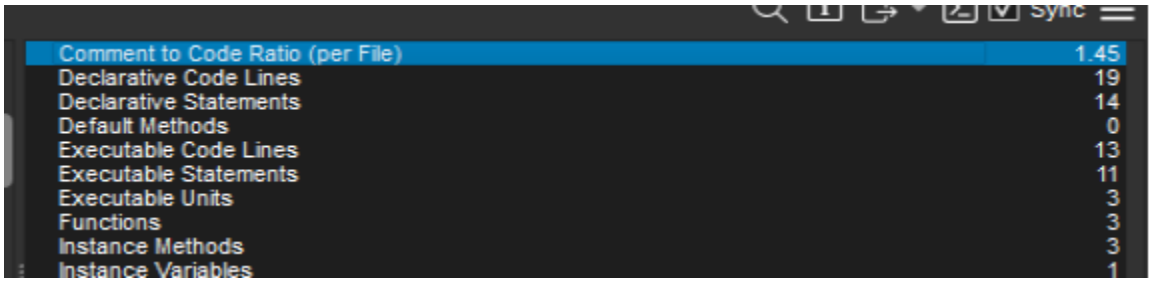
2. The plugin was successfully copied for customisation.



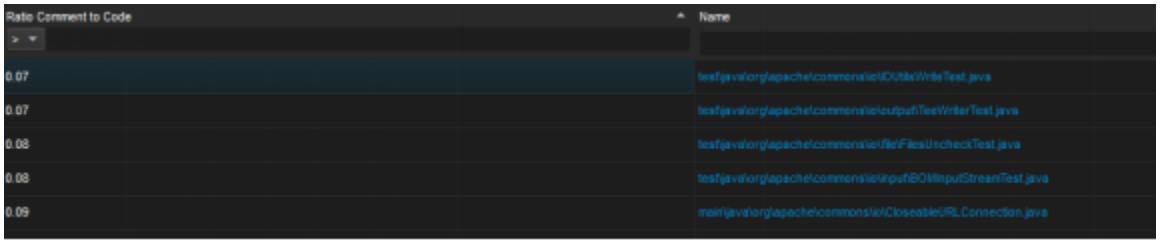
3. Changing the metric to fit the new formula



4. Metric printing in the browsing metric was successful.



5. The top five files with the lowest ratio of comments



Custom query in Understand

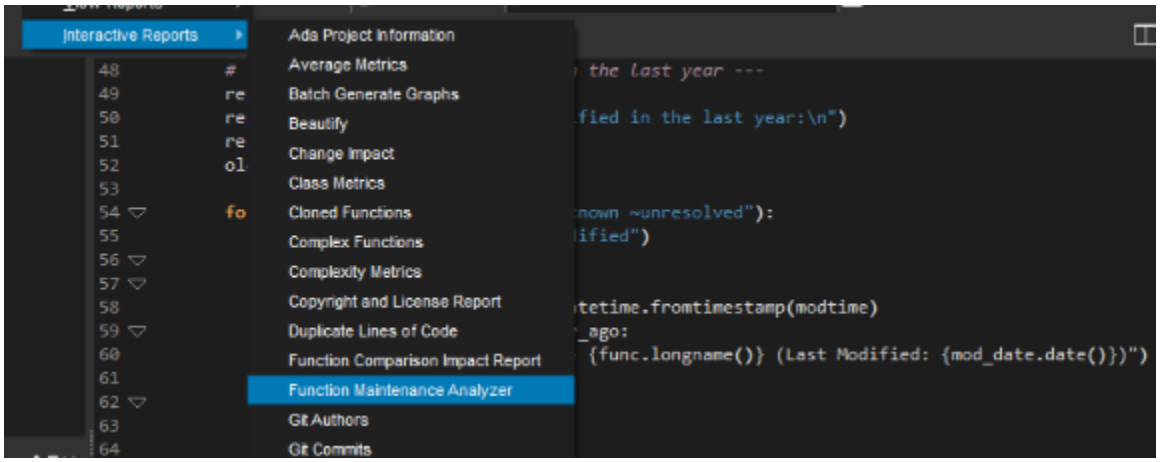
- **Functions Not Modified Recently:**
  These are functions that have stayed constant for over a year, suggesting they are either stable and reliable or perhaps outdated. Such routines should be checked periodically to ensure compatibility with newer modules and coding standards.
- **Dead Code Identification:**
  Functions that are never invoked or referenced anywhere in the source are considered dead code. These useless functions add code bloat, may mislead maintainers, and inflate code metrics. Removing or restructuring them improves code clarity, performance, and maintainability.

1. Writing query for function logic

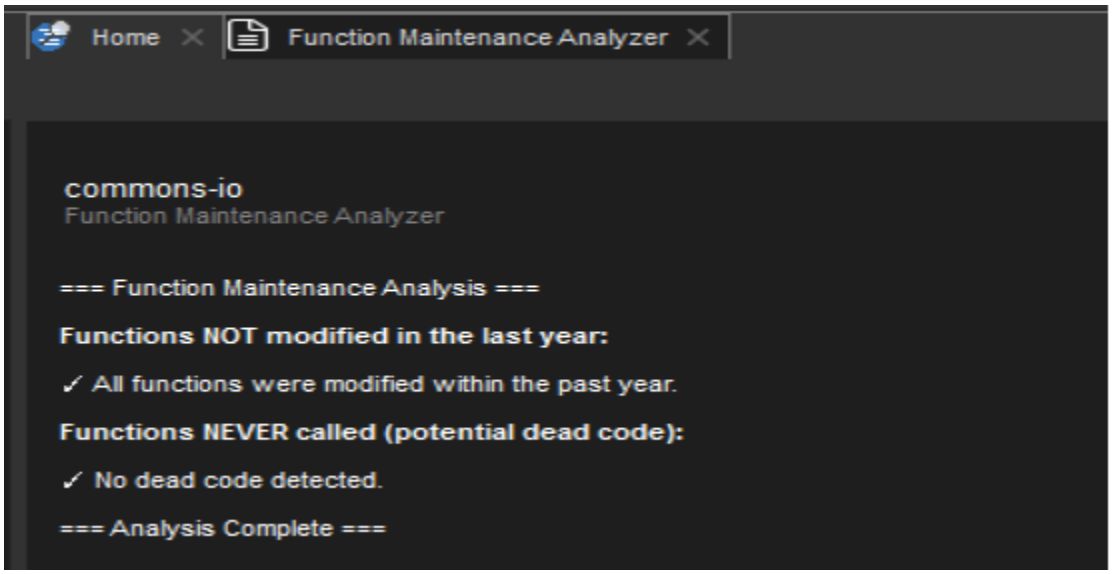
```
report.print("\nFunctions NOT modified in the last year:\n")
report.nobold()
old_funcs = 0

for func in db.ents("function ~unknown ~unresolved"):
    modtime = func.metric("TimeModified")
    if modtime:
        try:
            mod_date = datetime.datetime.fromtimestamp(modtime)
            if mod_date < one_year_ago:
                report.print(f"  - {func.longname()} (Last Modified: {mod_date.date()})")
                old_funcs += 1
        except Exception:
            pass

if old_funcs == 0:
    report.print("  ✓ All functions were modified within the past year.\n")

# --- 2. Dead Code Detection ---
report.bold()
report.print("\nFunctions NEVER called (potential dead code):\n")
report.nobold()
dead_funcs = 0

for func in db.ents("function ~unknown ~unresolved"):
    refs = func.refs("callby")
    if not refs:
        report.print(f"  - {func.longname()}")
        dead_funcs += 1
```

2. Running the query using run command

3. Going to report to see the query results



4. Viewing the results successfully



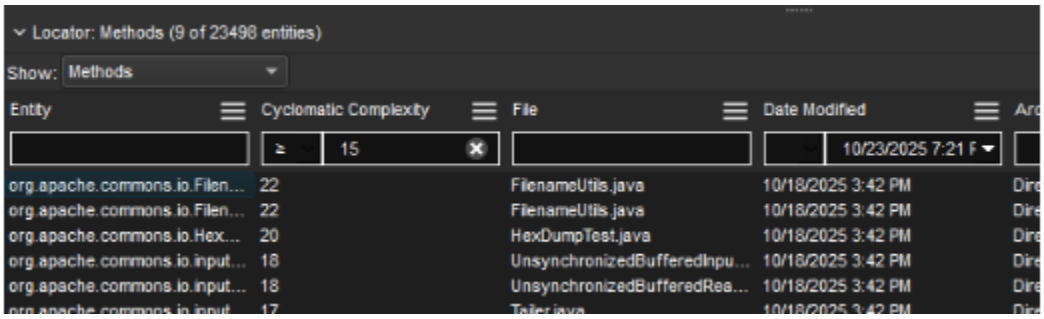
Summary of results

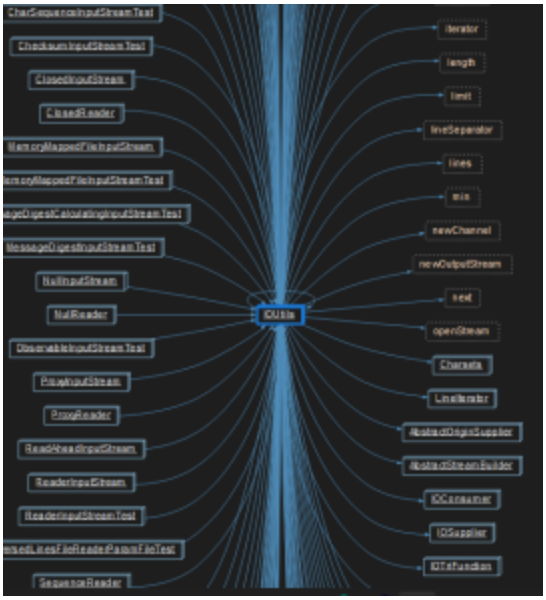
The findings showed that all files were edited last year and there are no functions that are never called.

7. Refactoring Suggestions

Identifying Refactoring Candidates

2 most complex methods and highly coupled methods are as follow:





Refactoring Strategy Recommendations

1. Class: IOUtils.java (High fan in and fan out)

Divide into Specialized Utility Classes: Break down existing IOUtils class into focused utility classes such as ByteIOUtils, CharIOUtils, CloseUtils, BufferUtils, ConvertUtils to promote modularity and decrease coupling.

Deprecate and Replace Legacy Methods: Gradually deprecate outdated or superfluous methods, especially those not charset-aware, and replace them with currentJava NIO APIs such `java.nio.file.Files`, `Channels`, and Java Streams for greater performance and maintainability.

Introduce Functional and Instance-Based Alternatives: Add a new IOHelper class that provides configurable, instance-based I/O processing. This allows developers to employ functional programming styles and easily adjust behavior for individual needs.

Improve Test Isolation and Internal Modularity: Make buffer sizes adjustable and transfer temporary buffer handling to specific internal helper classes. Use mocked IOUtils dependencies in unit tests to enhance reliability and isolate testing.

2. Class: FilesUtils.java (High fan in and fan out)

Break Down into Targeted Utility Classes: Divide the big FileUtils class into smaller, more focused classes like FileCopyUtils, FileDeleteUtils, FileReadUtils, FileWriteUtils, FileListUtils, and FileInfoUtils. By giving each class a specific duty, this division improves clarity, modularity, and ease of maintenance.

Adopt Modern Java NIO APIs: For better performance and error handling, swap out conventional `File` operations with `java.nio.file.Files` and associated APIs.
Deprecate out-of-date, charset-agnostic methods to conform to contemporary standards and introduce Path-based overloads for improved type safety.

Integrate Functional and Path-Based Enhancements: To handle files more neatly and expressively, use Streams and functional interfaces like `Predicate` and `Function`.
For further flexibility, introduce an instance-based FileHelper class that allows for customized file operations.

3. Function : getPrefixLength (High cyclomatic complexity (22))

Decompose Logic into Helper Methods: To clearly isolate duties and streamline code flow, divide path-handling logic into specific helper methods like handleHomePrefix(), handleDriveLetterPrefix(), and handleNetworkPrefix().

Use Guard Clauses Instead of Nested Conditions: Use guard clauses in place of deeply nested `if–else` structures to handle invalid or exceptional instances right away, resulting in a shorter and more readable technique.

Introduce Utility Method for Separator Checks: To improve clarity and minimize code duplication, provide a tiny helper such as isEitherSeparator(char) to simplify repeated separator validation logic.To eliminate redundancy and centralize index handling logic, move repetitious index resolution code into a separate method called resolveNotFound(int, int).

Expected Outcome: By reducing cyclomatic complexity from about 22 to 8–10, these refactorings would improve readability, modularity, and testability while simplifying future maintenance.

4. Function : doNormalize (High cyclomatic complexity (22))

Break Down Logic into Helper Methods: To isolate certain tasks and streamline overall flow, divide the method into smaller, targeted helpers like replaceOtherSeparators() and removeDuplicateSeparators().

Introduce Guard Clauses for Edge instances: Reduce the depth of nested conditions and improve readability by using guard clauses to handle faulty inputs or exceptional instances early.

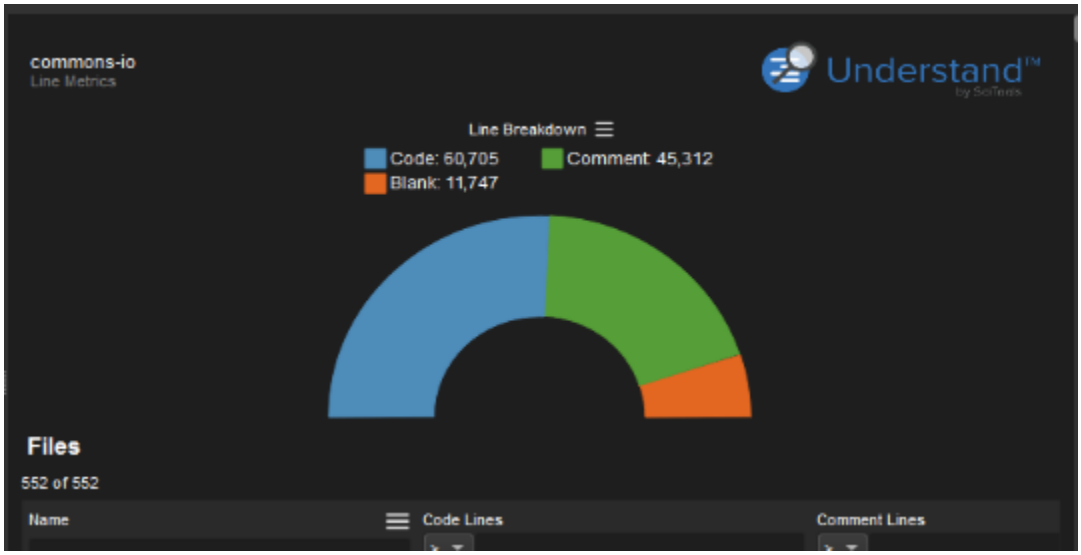
Separate Path Normalization Logic: To make the code more modular and simpler to test independently, extract the traversal handling and `../` resolution into a dedicated utility class or iterator.

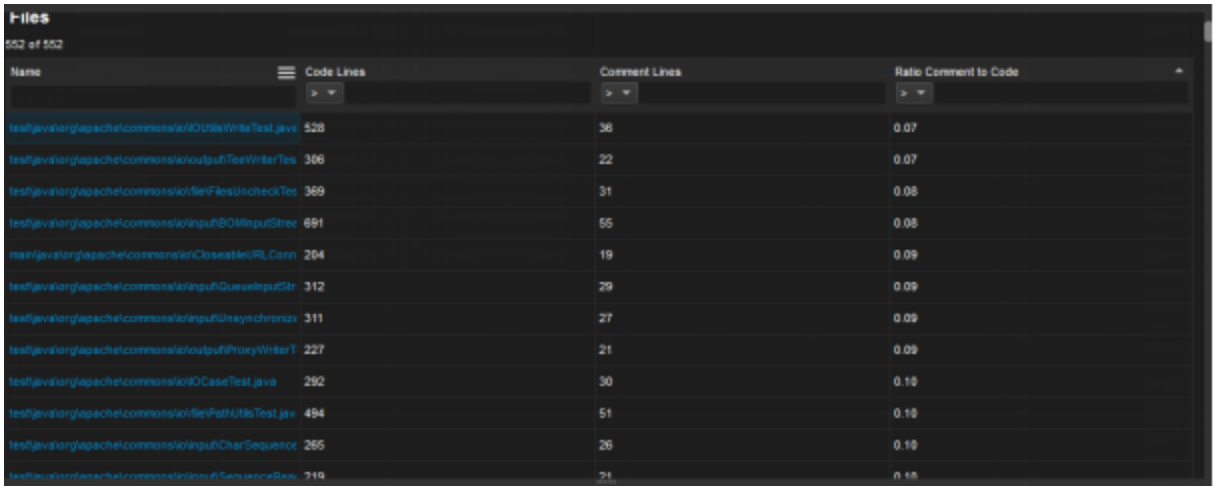
It is anticipated that these enhancements will lower the cyclomatic complexity to around 8–10, which will improve the code's modularity, readability, and maintainability.

8. Comment Analysis

Comment Density Report

- Total Code lines are 60,705
- Total Comment lines are 45,312





Identification of Under-Documented Areas

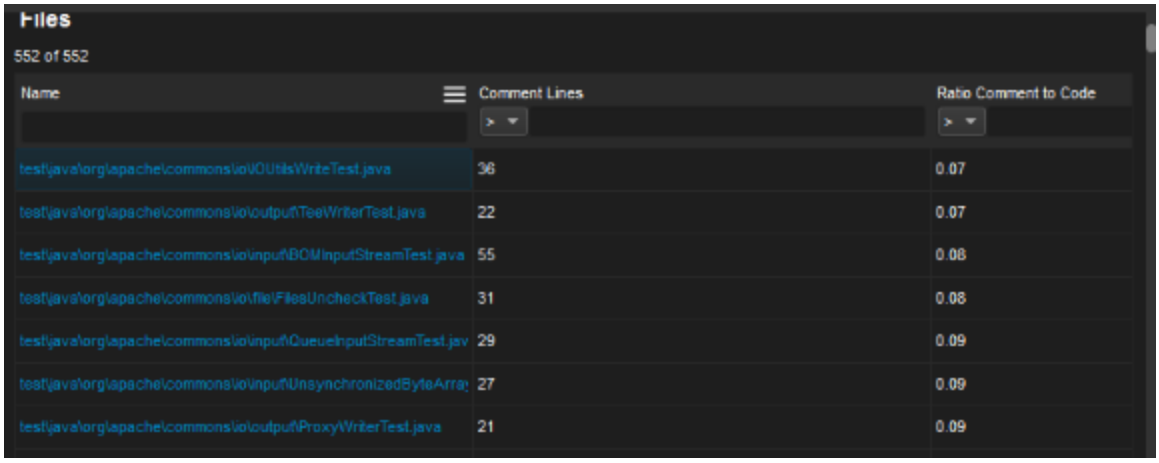
IOUtilsWriteTest.java: 36 comments (0.07%): More detailed notes on boundary scenarios are needed; there is not enough documentation on writer operation edge cases.

TeeWriterTest.java: 22 remarks (0.07%): The multi-output writing technique is explained very briefly; it should make clear how simultaneous stream writes are managed.

BOMInputStreamTest.java: 51 comments (0.08%): Needs more precise descriptions of encoding detection logic and the Byte Order Mark (BOM).

FileUncheckTest.java: 31 remarks (0.08%): Error-handling test cases are given very little information; more context would enhance comprehension.

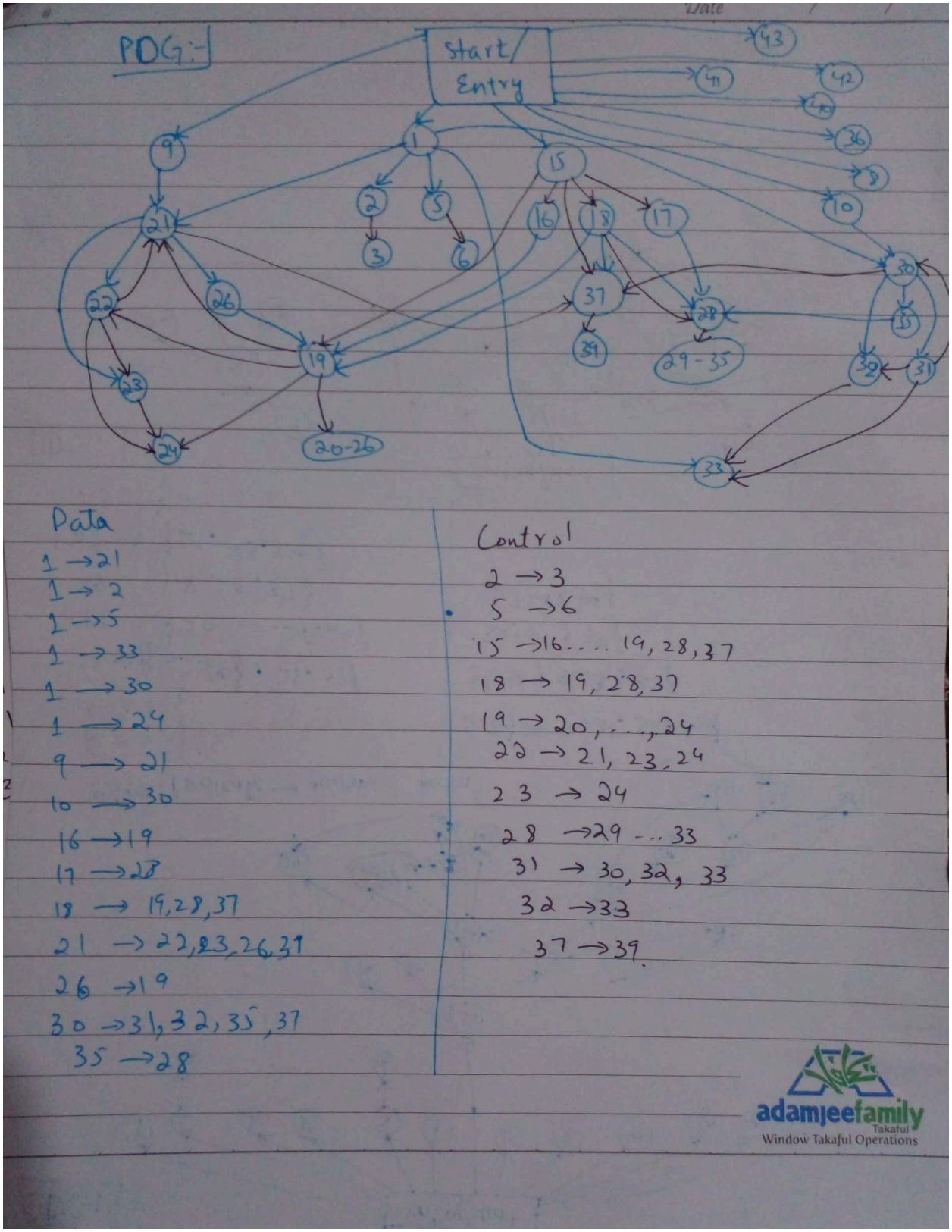
QueuedInputStreamTest.java: 29 comments (0.09%): Data flow and concurrency management should be covered in more detail in the documentation; threading and buffering behavior is lacking.



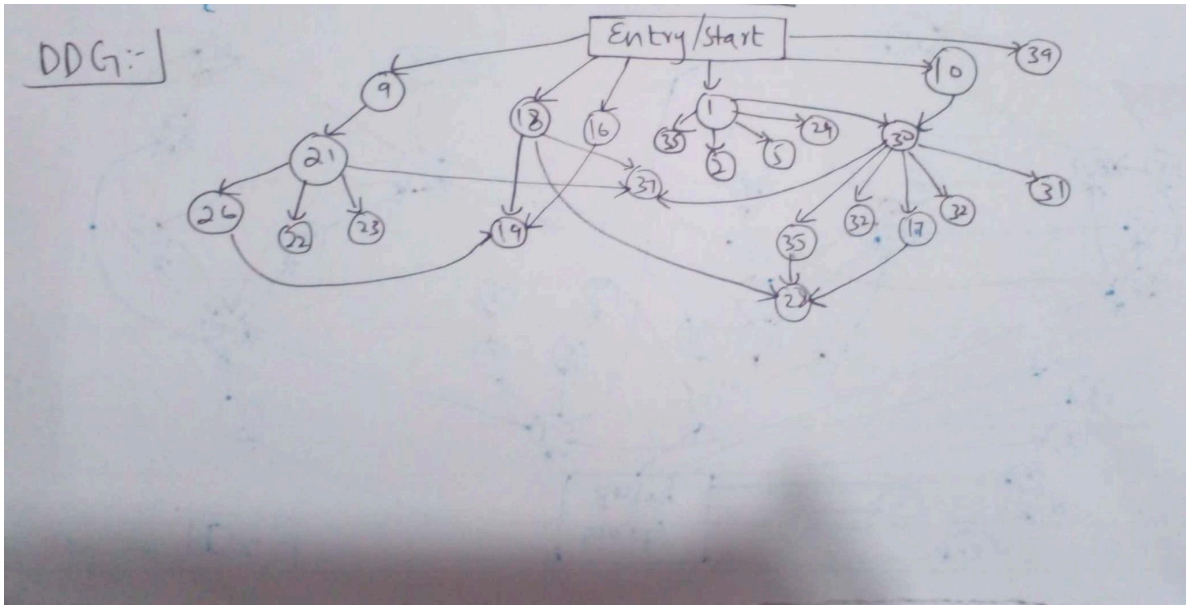
9. Manual Program Analysis

Following are graphs for method contentEquals(final Reader input1, final Reader input2)

CDG:


PDG:-
Start/
Entry
Data
1 → 21
1 → 2
1 → 5
1 → 33
1 → 30
1 → 24
9 → 21
10 → 30
16 → 19
17 → 28
18 → 19, 28, 37
21 → 22, 23, 26, 37
26 → 19
30 → 31, 32, 35, 37
35 → 28
Control
2 → 3
5 → 6
15 → 16 .... 19, 28, 37
18 → 19, 28, 37
19 → 20, ..., 24
22 → 21, 23, 24
23 → 24
28 → 29 ... 33
31 → 30, 32, 33
32 → 33
37 → 39
adamjeefamily
Takaful
Window Takaful Operations

DDG:

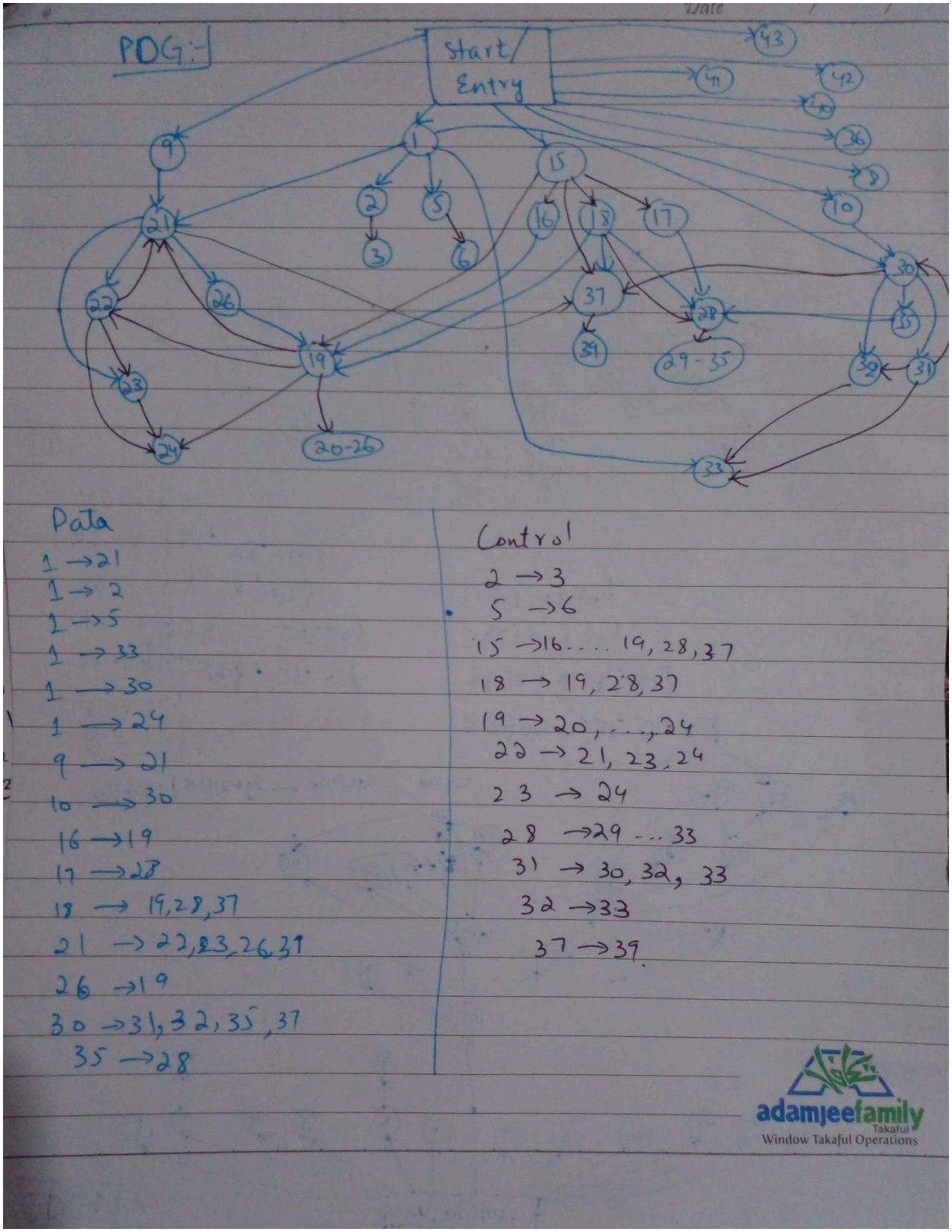


Def Node	Use
1	2, 5, 21, 24, 30, 33
9	21
10	30
16	19
17	28
21	22, 23, 26, 37
26	19
30	31, 32, 35, 37
35	28

Def - use Pair

(1,2), (1,5), (1,21), (1,24), (1,30), (1,33)
(9, 21)
(10, 30)
(16, 19)
(17, 28)
(21,22), (21,23), (21,26), (21,37)
(26, 19)
(30,31), (30,32), (30,35), (30,37)
(35, 28)

PDG:


PDG:-
Start/
Entry
43
41
42
36
8
10
30
35
32
31
33
9
1
15
2
5
16
18
17
3
6
21
22
26
37
28
39
29-35
23
19
24
20-26
Data
1 → 21
1 → 2
1 → 5
1 → 33
1 → 30
1 → 24
9 → 21
10 → 30
16 → 19
17 → 28
18 → 19, 28, 37
21 → 22, 23, 26, 37
26 → 19
30 → 31, 32, 35, 37
35 → 28
Control
2 → 3
5 → 6
15 → 16 .... 19, 28, 37
18 → 19, 28, 37
19 → 20, ..., 24
22 → 21, 23, 24
23 → 24
28 → 29 ... 33
31 → 30, 32, 33
32 → 33
37 → 39
adamjeefamily
Takaful
Window Takaful Operations